

基于热质耦合与随体收缩模型的药材烘干预测

摘 要

热风烘干中，药材内部升温与水分迁移存在不同时间尺度，含水率变化又会改变物性；半径收缩后，还需区分计算区域的几何积分和以干固体为分母的真实水质量。本文针对圆柱药材的预热、全程干燥及收缩过程，建立与题目四问对应的径向热质耦合模型，并对后期边界延续、运动描述和端面影响分别检验。

对固定圆柱，以散度形式建立导热与含水率扩散方程，使用 PCHIP 构造附件观测区间内的空气边界。采用圆柱有限体积离散、隐式 Picard 迭代和非线性界面求积。对收缩圆柱，以规定半径构造随体归一化坐标，在均匀径向形变和独立干固体守恒假设下推导水分方程，将题目密度与比热容的乘积用于有效体积热容；4 h 后推荐使用末一小时观测均值平台。

问题一在 1800 s 时，中心与表面温度分别为 33.5764 和 36.7861 °C，表面干基含水率为 1.5104 kg/kg。问题二在 3 h 时中心温度已为 49.8494 °C，但最大含水率仍为 1.7662 kg/kg。问题三在附录 3 物性、固定半径和末点延续下，达标时间为 57.1726 h；问题四在附录 4 物性、随体收缩和均值平台下，推荐时长为 51.0913 h。问题四末点及名义平台分别得到 50.8235 和 51.0899 h，作为边界情景保留。

同物性、同平台的固定半径对照为 129.8561 h，说明不能把问题三与四的差值全部归因于收缩。问题四网格由 81 增至 161 节点时，达标时间改变 6.56 s；步长由 1 降至 0.5 s 时改变 1.19 s。单参数试验表明扩散系数影响最强。轴对称端面对照未分辨出配对达标时差，但 4 h 平均含水率降低约 3.11%，故一维近似的适用性应按预测指标区分。本文结论适用于明确的经验物性、材料运动和空气边界假设。

关键词：圆柱药材；热风烘干；有限体积法；材料坐标；干基含水率

1 问题重述与分析

1.1 问题背景与研究任务

热风烘干通过外部空气向药材供热，并将药材内部迁移至表面的水分带走。表面先接触热风，内部升温与失水存在滞后；随着含水率降低，导热能力、储热能力和水分扩散能力也会变化。因此，外部空气接近恒温，并不表示药材已经干燥，更不能仅凭表面含水率判断整个药材是否达标。题目给出的初始药材近似为长 25 cm、半径 2 cm 的圆柱，初始温度为 28 °C，初始干基含水率为 2.55 kg/kg，烘房初期温度及水分变量见附件 1，烘干过程的半径变化见附件 2。

问题一要求采用附录 2 的参数，求出前 1800 s 的温度场和含水率场，并按指定时间与径向位置给出结果。问题二改用附录 3 的经验物性，建立适用于整个过程的热质传递模型，输出前 3 h 的温湿分布。问题三在问题二的物性模型上，确定所有位置含水率均低于 0.15 kg/kg 的首次时刻，并给出全程水分分布。问题四进一步使用附件 2 的变化半径和附录 4 的经验公式，确定收缩药材的干燥时长。四问均需按附件模板输出完整 Excel 结果。

1.2 模型选择与需要区分的量

本文采用圆柱径向热质耦合模型，以局部温度和干基含水率为状态变量。选择空间分布模型的直接原因是题目同时规定中心、内部和表面的输出位置。一个平均温度或平均含水率的常微分方程无法恢复这些剖面，也不能直接满足“各处达标”的判据。对初始尺度，端面总面积与侧面积之比为 $R_0/L = 0.08$ ，说明侧向传递具有较大的交换面积，但这一面积比本身不是一维近似的误差上界。本文先建立径向主模型，再以轴对称二维计算检验端面作用。

问题四的关键是明确网格所代表的对象。半径随时间变小，可以描述材料整体收缩，也可以只表示在静止场上截取的计算区域缩小；两者的坐标公式相似，物理质量预算却不同。本文将 $\xi = r/R(t)$ 规定为材料坐标，使同一 ξ 对应同一干固体材料点，并从干固体和水的连续方程推导含水率方程。一般的任意拉格朗日-欧拉描述需要区分材料速度与网格速度^[1]，这里采用二者相等的特例。

另一个关键是数据覆盖范围。附件 1 只覆盖 0 至 4 h，而达标时刻超过 50 h。因此，4 h 以后的空气边界不能被称为附件实测值。本文保留已有固定半径结果所采用的末点延续作为问题三的明确情景；问题四推荐采用末一小时观测均值平台，并同时计算末点平台和名义平台。两问的物性与平台不同，不能把它们的时间差全部归因于收缩。收缩作用另由同物性、同平台的配对计算估计。

2 模型假设、符号与数据处理

2.1 模型假设及其适用范围

假设药材在初始时具有轴对称、均匀的温度和含水率，热风在侧面形成空间均匀、随时间变化的边界。主模型忽略周向和轴向差异；关于轴向近似的适用程度，由第9节的二维对照单独判断。问题一至三保持圆柱半径和长度不变，问题四保持长度不变，半径按附件2规定，并采用均匀的径向随体形变。附件只给出外轮廓半径，未给局部位移和孔隙信息，故均匀形变是模型闭合所用的假设，而不是从半径曲线唯一推得的事实。

假设干固体不发生反应和损失，以独立干固体密度 s 记录其质量。固定半径时 s 为空间和时间常量；均匀径向收缩时 s 在空间上均匀，随体积改变而变化。题目给出的 $\rho(C)$ 、 $c_p(C)$ 、 $k(C)$ 及 $D(C, T)$ 均按相应问题使用。尤其将 $\rho(C)c_p(C)$ 解释为温度方程的经验有效体积热容，记作 $a(C)$ ，不再额外强制 $\rho(C)/(1+C)$ 等于守恒干固体密度。这个区分对收缩模型不可省略，其一致性检验见第7.3节。

假设水分相对于干固体的迁移可由有效扩散通量表示，表面采用题目系数构成的经验 Robin 传质边界。用 C_{eq} 表示与空气状态对应的等效表面平衡含水变量，并取 $C_{eq} = C_{air}$ 。由于附件没有吸附等温线和分配系数，这一关系属于有效经验映射；它不意味着空气与药材两种 kg/kg 变量具有同一个质量分母。

温度与含水率通过经验物性相互作用，不引入题目未给定的相变潜热、压力驱动和化学反应参数。因此，温度方程是给定资料下的有效导热近似，不是包含全部潜热与多相焓输运的能量守恒模型。本文不据此计算能耗，也不把数值收支误差小解释为真实工艺已经获得实验验证。

2.2 主要符号与单位

全文采用 SI 单位计算，制表时将时间转换为题目指定的秒或小时，将径向距离转换为厘米。温度场以 T 表示，单位为 $^{\circ}\text{C}$ ；进入指数经验公式时使用绝对温度 $\Theta = T + 273.15$ ，单位为 K。温差在两种温标下数值相同，但指数中的温度不能直接使用摄氏数值。

r, ξ r 为距圆柱轴线的径向距离，单位 m； $\xi = r/R(t) \in [0, 1]$ 为归一化径向坐标。固定半径模型中也使用此网格记法。

R, L $R(t)$ 为当前半径， $R_0 = 0.02 \text{ m}$ ； $L = 0.25 \text{ m}$ 为固定长度。二维对照使用轴向半长度 $H = L/2$ 。

C 干基含水率，定义为水质量与干固体质量之比，单位 kg 水/kg 干固体。 C_s 为侧表面值， C_{max} 为计算域内最大值。

s, v, w s 为单位当前体积内的干固体质量，单位 kg/m^3 ； v 为干固体径向速度， w 为网格径向速度，单位 m/s。

a, k, D $a = \rho_{eff}c_p$ 为有效体积热容，单位 $\text{J}/(\text{m}^3 \text{ K})$ ； k 为导热系数，单位 $\text{W}/(\text{m K})$ ； D 为

水分有效扩散系数，单位 m^2/s 。

h_T, h_m $h_T = 25 \text{ W}/(\text{m}^2 \text{ K})$ 为换热系数； $h_m = 8 \times 10^{-7} \text{ m/s}$ 为经验传质系数。三套物性模型均使用这两个系数。

$t_*, \bar{C}$ t_* 为全域最大含水率首次跨过阈值的时间； $\bar{C} = M_w/M_d$ 为以总干质量归一化的平均含水率。

2.3 观测区间内的插值

附件 1 包含 241 个观测时刻，间隔为 60 s，范围为 0 至 14400 s；附件 2 包含 145 个半径观测值，间隔为 1800 s，范围为 0 至 259200 s。两份数据均保留原始采样值，不为了获得光滑图像而平滑或改写平台波动。求解时，对温度、空气水分变量和半径分别使用分段三次 Hermite 保形插值（PCHIP）。这一类局部插值通过邻近割线确定节点斜率，使分段单调数据在区间内保持单调^[2]，适合处理升温段与接近平稳的平台段。其作用是构造观测点之间的连续输入，不是增加观测信息。

图1显示温度在初期持续升高，后段接近 50°C ；空气水分变量则逐渐接近 0.05。采用线性插值的替代计算用于检查插值选择对问题四达标时间的影响，结果在第8.4节给出。本文将采样分辨率、积分时间步长和结果输出间隔分别处理：60 s 采样并不限制数值时间步长只能取 60 s，1 s 输出也不意味着计算误差为 1 s。

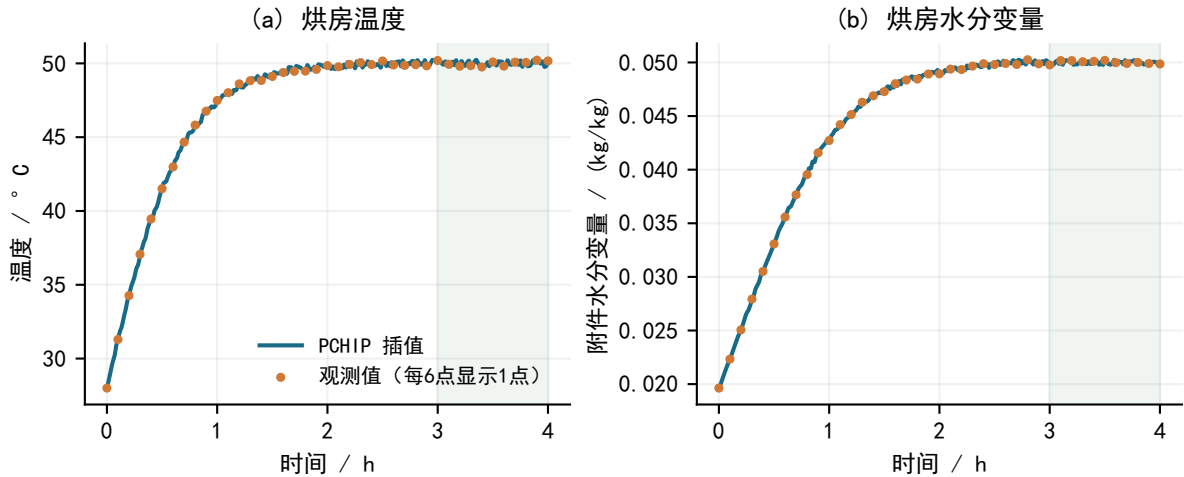


图 1 附件 1 的烘房观测与区间内插值。浅色区域为用于计算平台均值的 3 至 4 h；图中散点为原始观测的等间隔显示子集。

图2给出附件 2 的半径曲线及其在归一化方程中的几何倍率。半径在前 12 h 由 2 cm 减至 1.248 cm，之后变化明显放缓；36 h 至问题四终点的记录约为 1.200 cm。主结果的 51.09 h 位于半径实测区间内，因而计算不需要把半径外推到 72 h 以后。图中几何倍率仅表示在同一物性值下，归一化扩散和表面交换系数随尺度的变化，不能独立代替非线性过程的求解。

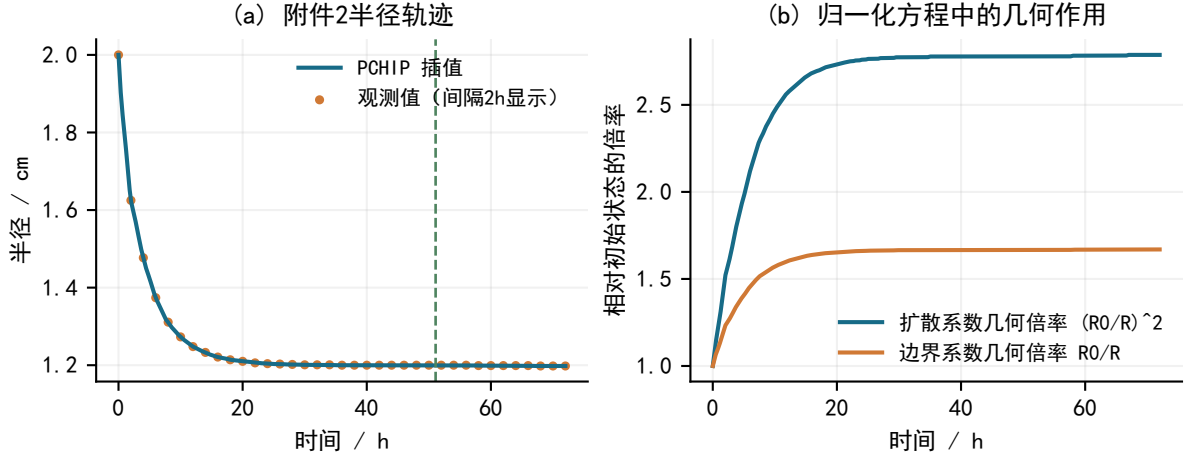


图2 规定半径及其几何作用。左图虚线标记问题四达标时刻；右图中的倍率相对于初始半径计算。

2.4 4 h 以后的边界延续

末一小时包含 3 h 和 4 h 两个端点，共 61 个观测值。由于采样间隔相等，本文以这些值的算术平均构造恒定平台：

$$T_{\text{air,mean}} = 49.9989344262 \text{ }^{\circ}\text{C}, \quad C_{\text{eq,mean}} = 0.049987540984. \quad (1)$$

这一平台使用了后段全部采样信息，降低了单个末点波动对长期延续的影响。它与名义的 50 °C、0.05 平台接近，因而作为问题四的推荐情景。另一方面，附件未明确规定 4 h 后的工艺控制，此推荐仍然是一项延续假设，不能称作已知边界条件。

另外保留两种情景：末点保持为 50.165 °C 和 0.04986；名义平台为 50 °C 和 0.05。所有情景在 0 至 4 h 完全使用同一插值，只有 $t > 4 \text{ h}$ 时采用各自的恒定值。问题三主结果使用末点保持；问题四主结果使用式(1)。均值平台在 4 h 右侧可能相对末次采样产生一个小跳变，数值计算按分段定义直接处理，没有虚构一段过渡观测。

3 固定圆柱热质传递模型与数值方法

3.1 控制方程及初边值条件

在半径固定的圆柱内，温度与水分的径向模型为

$$a(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k(C) \frac{\partial T}{\partial r} \right), \quad (2)$$

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C, \Theta) \frac{\partial C}{\partial r} \right). \quad (3)$$

式(3)对应独立、均匀且不变的干固体密度。若写成真实水质量方程，其储存项与扩散通量均乘以 s ，因 s 为空间常量而可约去。由此得到的体积加权含水率也等于干质量加权含水率。这里的质量解释不依赖热学有效密度的数值变化。

轴线处的对称条件和侧面的交换条件为

$$T_r(0, t) = 0, \quad C_r(0, t) = 0, \quad (4)$$

$$-kT_r(R_0, t) = h_T(T_s - T_{\text{air}}), \quad -DC_r(R_0, t) = h_m(C_s - C_{\text{eq}}). \quad (5)$$

这里将朝圆柱外的通量定义为正。预热时常有 $T_s < T_{\text{air}}$ ，故热通量为负，表示热量流入药材；干燥时 $C_s > C_{\text{eq}}$ ，水分通量为正，表示水向外迁移。这一符号约定同时用于边界离散与整体收支，避免将升温方向或排水方向写反。初始条件为

$$T(r, 0) = 28^\circ\text{C}, \quad C(r, 0) = 2.55, \quad 0 \leq r \leq R_0. \quad (6)$$

注意 D 、 k 均放在散度内部。含水率不均匀时，它们在空间中变化，因此不能将 $\partial_r(rDC_r)/r$ 直接替换为 $D(C_{rr} + C_r/r)$ 而遗漏系数梯度。有效体积热容乘以局部温度变化率；本文没有将其改写为 $\partial_t(aT)$ ，后者会额外引入 a_tT 项，属于另一种热学闭合。

3.2 三套经验物性的一致使用

问题一仅采用附录 2：

$$a = 820 \times 2600, \quad k = 0.36, \quad D_1(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right). \quad (7)$$

问题二和问题三从初始时刻起统一采用附录 3，而不是在预热结束后才切换：

$$\begin{aligned} \rho_{\text{eff},23} &= 650 + 128C, & c_{p,23} &= 1450 + 2736 \frac{C}{C+1}, \\ k_{23} &= 0.21 + 0.38 \frac{C}{C+1}, & D_{23} &= 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C} - \frac{3850}{\Theta}\right). \end{aligned} \quad (8)$$

问题四使用附录 4：

$$\begin{aligned} \rho_{\text{eff},4} &= 760 + 90C, & c_{p,4} &= 1850 + 2150 \frac{C}{C+1}, \\ k_4 &= 0.12 + 0.20 \frac{C}{C+1}, & D_4 &= 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{C} - \frac{3850}{\Theta}\right). \end{aligned} \quad (9)$$

上述密度、比热容、导热系数和扩散系数的单位依次为 kg/m^3 、 $\text{J}/(\text{kg K})$ 、 $\text{W}/(\text{m K})$ 和 m^2/s ， $a_q = \rho_{\text{eff},q}c_{p,q}$ 。物性表达式只按题号选择，不能用问题一的常数热物性去生成问题二的前半小时结果，也不能将问题三的参数和问题四的收缩混合后称为题目指定的第四问模型。

这些公式使系统具有双向的参数耦合：温度进入 D ，影响含水率变化；含水率又进入有效体积热容和导热系数，影响升温。对于 $D = A \exp(-b/C - 3850/\Theta)$ ，直接求导有

$$\frac{\partial \log D}{\partial C} = \frac{b}{C^2} > 0, \quad \frac{\partial \log D}{\partial \Theta} = \frac{3850}{\Theta^2} > 0. \quad (10)$$

因此，在其他条件相同的情况下，升温提高扩散能力，而失水降低扩散能力。干燥后期两种作用不再对称：温度已经接近平台，但低含水率仍使扩散系数下降，形成后期的缓慢失水段。这也说明用初始扩散系数估计全程达标时间缺少依据。

3.3 圆柱控制体及界面通量

有限体积法从每个控制体的收支出发，将相邻单元间的同一通量以相反符号加入两侧，从而实现离散局部守恒^[3]。本文取 N 个节点 $\xi_i = i/(N-1)$, $i = 0, \dots, N-1$ ，首末节点分别位于轴线与表面；相邻节点中点为内部控制面，边界控制面为 0 和 1。记控制面为 $\xi_{i\pm 1/2}$ ，则径向权重为

$$V_i = \frac{\xi_{i+1/2}^2 - \xi_{i-1/2}^2}{2}, \quad \sum_i V_i = \frac{1}{2}. \quad (11)$$

首节点仍具有有限的半控制体，离散时无需在 $r = 0$ 直接计算 $1/r$ 。表面也保留半控制体，而不是把边界节点当作无容量的代数节点。这样既避免轴线奇点，又能用同一组权重进行全局水分预算。

设 U 代表温度或含水率， p 代表相应的导热或扩散系数。内部相邻节点的传递系数为

$$G_{i+1/2} = \frac{\xi_{i+1/2}}{\Delta\xi} p_{i+1/2}, \quad \Delta\xi = \frac{1}{N-1}. \quad (12)$$

热传导使用正的调和界面均值 $k_{i+1/2} = 2k_i k_{i+1} / (k_i + k_{i+1})$ 。水分扩散对低含水率十分敏感，直接平均两个端点的 D 可能不足以解析表面的干燥薄层。本文在线性重构的状态路径上计算

$$D_{i+1/2} = \int_0^1 D((1-\eta)C_i + \eta C_{i+1}, (1-\eta)\Theta_i + \eta\Theta_{i+1}) d\eta, \quad (13)$$

并用 8 点 Gauss-Legendre 正权重求积实现。恒温时，式(13)等价于 Kirchhoff 势对 C 的差商；温度同时变化时，它是沿重构状态路径的有效界面系数，本文不将其称作完整非线性边值问题的精确通量。该系数保持正值，因而不会因指数非线性引入负的数值扩散。独立自适应求积与此 8 点规则的比较见第 8.3 节。

3.4 隐式迭代与守恒预算

将热方程容量记为 $b_i = a_i$ 、水分方程容量记为 $b_i = 1$ 。在固定半径或材料坐标形式下，后向 Euler 离散可统一写为

$$b_i R_n^2 V_i \frac{U_i^n - U_i^{n-1}}{\Delta t} = G_{i+1/2} (U_{i+1}^n - U_i^n) - G_{i-1/2} (U_i^n - U_{i-1}^n) - \mathbf{1}_{i=N-1} R_n h (U_i^n - U_\infty^n), \quad (14)$$

轴线和表面之外的内部面项均取零； R_n 为当前步半径，固定模型中始终为 R_0 。这里 h 分别为 h_T 和 h_m 。在随体模型中， R_n^2 同时乘在新旧状态差上；不能只在新时刻乘 R_n^2 而将旧时刻改用 R_{n-1}^2 ，否则将改变所离散的质量变量。

每个时间步以旧时刻状态作为迭代初值，按当前 T, C 更新容量与界面系数，分别解两个三对角线性系统，再用所得新场更新物性，直至

$$\|T^{(m+1)} - T^{(m)}\|_\infty < 10^{-7} \text{ }^\circ\text{C}, \quad \|C^{(m+1)} - C^{(m)}\|_\infty < 10^{-9}. \quad (15)$$

三对角系统使用 Thomas 算法。对主结果，迭代不作欠松弛，四问各步实际最多迭代 3 次；程序还检查非有限值、非正含水率和迭代不收敛，不将这些情况作为可接受输出。每次线性求解的计算量随节点数线性增加，适合长时间的逐步推进。

主结果统一采用 $N = 81$ 、 $\Delta t = 0.5 \text{ s}$ 、后向 Euler。独立比较使用二阶后向差分 BDF2，其时间导数为 $(3U^n - 4U^{n-1} + U^{n-2})/(2\Delta t)$ ，第一步以 Euler 启动。两者使用相同的空间通量和物性更新，从而把时间离散选择与模型结构选择分开。

对随体含水率方程，式(14)在所有节点求和后，内部通量严格抵消，得到

$$\sum_i V_i (C_i^n - C_i^{n-1}) = -\Delta t \frac{h_m}{R_n} (C_s^n - C_{eq}^n). \quad (16)$$

本文同时检查逐步与累计收支。该预算所守恒的是相对于固定干质量的含水量，热学有效密度没有出现在质量分母中。数值预算与网格收敛回答不同问题：前者检验离散通量和储存项是否一致，后者检验特定预测量对离散参数是否敏感；二者都不能单独验证材料运动假设。

4 问题一：预热平衡阶段的温湿分布

4.1 参数设置与计算结果

将式(7)代入固定圆柱模型，采用附件 1 在 0 至 1800 s 的插值边界。该问题的温度方程为常系数，但水分扩散仍依赖局部含水率，所以不能把整个问题当作两个常系数扩散方程。计算结果按实际 r 位置插值到题目要求的 0、0.5、1、1.5、2 cm，并保留四位小数。

表1和表2分别对应题目要求的温度和含水率结果。表面首先升温，随后温度扰动向中心传递。100 s 时中心温度仍为 28.0001 °C，表面已达到 28.1797 °C；1800 s 时中心和表面分别为 33.5764 °C、36.7861 °C，中心较表面低 3.2097 °C。水分的空间滞后更强：终点表面含水率已下降到 1.5104，而中心仍为 2.5500（按四位小数显示）。

表 1 30 分钟内药材的温度（单位：°C）

时间 / s	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
100	28.0001	28.0003	28.0040	28.0321	28.1797
300	28.0410	28.0638	28.1517	28.3677	28.8489
600	28.4542	28.5369	28.8046	29.3161	30.1645
900	29.3253	29.4592	29.8765	30.6170	31.7300
1200	30.5438	30.7108	31.2233	32.1136	33.4299
1500	31.9968	32.1878	32.7674	33.7481	35.1215
1800	33.5764	33.7730	34.3648	35.3626	36.7861

表 2 30 分钟内药材的干基含水率 (单位: kg/kg)

时间 / s	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
100	2.5500	2.5500	2.5500	2.5500	2.2491
300	2.5500	2.5500	2.5500	2.5491	2.0518
600	2.5500	2.5500	2.5500	2.5352	1.8775
900	2.5500	2.5500	2.5497	2.5045	1.7551
1200	2.5500	2.5500	2.5481	2.4646	1.6588
1500	2.5500	2.5499	2.5445	2.4206	1.5789
1800	2.5500	2.5497	2.5382	2.3755	1.5104

4.2 温湿变化的尺度解释

图3给出了四个代表时刻的剖面。温度曲线在整个径向范围内逐渐变化, 含水率曲线则首先在靠近表面的区域形成较大梯度。这里中心含水率显示为 2.5500 并不代表数值方程没有执行水分扩散, 而是当前时长内中心变化低于表格的显示分辨率。求解保留全精度, 最终中心值约为 2.54999175。

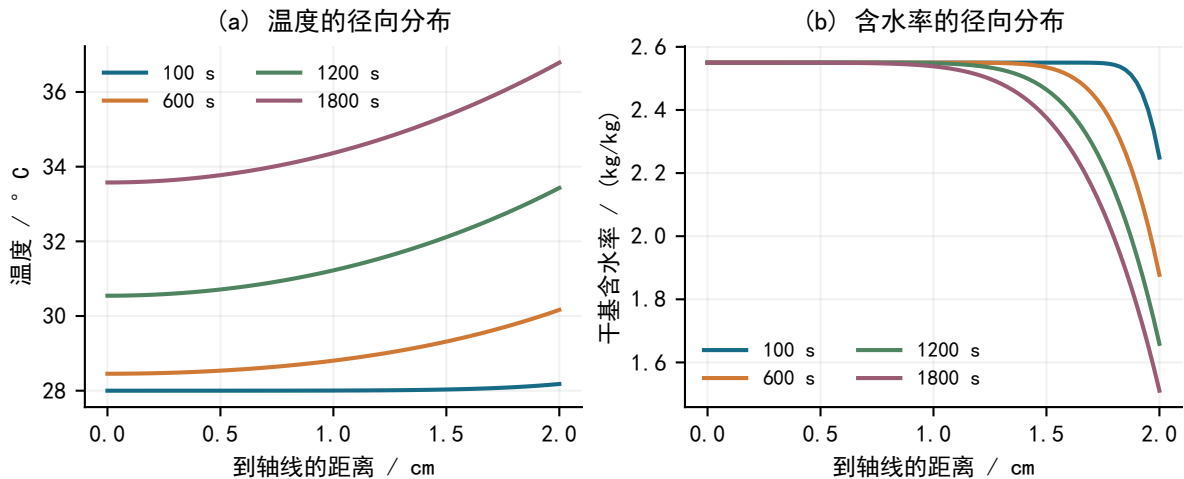


图 3 问题一的径向温度与含水率剖面。温度采用常数热物性, 水分采用随含水率变化的扩散系数。

用初始物性作尺度估计, 热扩散率为 $\alpha_1 = k/(\rho c_p) \approx 1.69 \times 10^{-7} \text{ m}^2/\text{s}$, 而初始水分扩散系数约为 $4.94 \times 10^{-9} \text{ m}^2/\text{s}$ 。二者相差一个数量级以上。在 1800 s 内, $\sqrt{\alpha_1 t}$ 约为 1.74 cm, $\sqrt{D_1(C_0)t}$ 约为 0.30 cm; 后者还是未考虑近表面失水导致 D 下降的初始尺度。这个估计解释了热影响已传播至中心、水分变化仍集中于表层的趋势, 但不替代非线性剖面的定量计算。

初始热 Biot 数为 $h_T R_0/k \approx 1.39$, 不足以支持把圆柱温度近似为处处相同的集中参数模型。表1中的中心和表面差异与这一判断一致。完整结果以 1 s 间隔输出, 径向间隔 0.1 cm, 分别写入 `result1.xlsx` 的“温度”和“水分浓度”工作表; 其输出采样不同于内部 0.5 s 积分步长。

5 问题二：经验物性下的全程耦合模型

5.1 模型实施与前 3 小时结果

问题二保持固定几何与初边值条件，全部物性改用式(8)。温度影响水分扩散，水分反过来改变有效容量和导热系数；通过每一步内的 Picard 迭代共同求解两个场。按照题目“统一采用附录 3”的要求，0 至 0.5 h 也使用这套经验式，而不是拼接问题一的轨迹。因此，同为 1800 s，问题二与问题一得到不同的温度和表面含水率，是模型输入不同所致。

表3和表4给出 0.5 h 间隔的结果。0.5 h 时中心温度 32.1904 °C、表面温度 35.4137 °C；3 h 时中心升至 49.8494 °C、表面 49.9666 °C，径向温差已降至约 0.1172 °C。与此相比，3 h 时中心含水率仍为 1.7662，表面为 1.0081，均明显高于 0.15 的要求。这说明温度接近空气平台只表明热响应接近稳定，并不等同于完成干燥。

表 3 3 小时内药材的温度（单位：°C）

时间 / h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
0.5	32.1904	32.3832	32.9667	33.9614	35.4137
1	40.3822	40.5542	41.0609	41.8784	43.0001
1.5	45.8468	45.9348	46.1930	46.6052	47.1396
2	48.4501	48.4879	48.5983	48.7734	49.0032
2.5	49.4669	49.4791	49.5135	49.5645	49.6617
3	49.8494	49.8553	49.8746	49.9101	49.9666

表 4 3 小时内药材的干基含水率（单位：kg/kg）

时间 / h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
0.5	2.5499	2.5489	2.5255	2.3258	1.6487
1	2.5257	2.4947	2.3578	2.0230	1.4711
1.5	2.3860	2.3256	2.1344	1.8020	1.3476
2	2.1708	2.1084	1.9236	1.6259	1.2311
2.5	1.9566	1.9006	1.7360	1.4720	1.1166
3	1.7662	1.7165	1.5702	1.3333	1.0081

图4展示了这一差异：温度曲线在升温过程中整体上移并逐渐变平，含水率剖面虽整体下降，仍保持中心较高、表面较低的空间分布。完整数据按 1 s 和 0.1 cm 输出到 result2.xlsx，温度和含水率分别存入对应工作表。

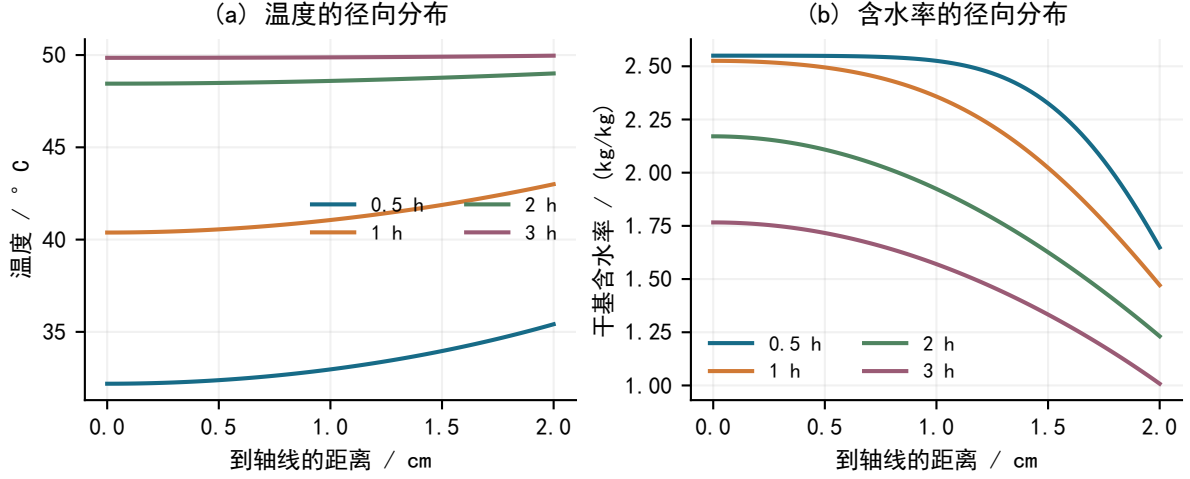


图4 问题二采用附录3物性时的前3小时剖面。前半小时同样使用附录3，未与问题一的参数拼接。

5.2 耦合的方向与后期减速机制

由式(10)，升温 and 失水对扩散系数产生方向相反的作用。初期升温使迁移加快，同时表层含水率下降使其扩散能力变弱；随后药材接近恒温，温度提高扩散的作用基本停止，而低含水率层仍会阻碍内部水分外移。式(8)还表明 k 、 c_p 随 C 下降而下降，因此内部温度的响应也依赖干燥进程。

这种参数耦合与加入额外水分输运热源是不同的建模选择。本文的温度场来自有效导热方程，并未在其中暗含一个未标定的蒸发冷却系数。若要利用模型进一步预测真实耗能或表面蒸发温差，还需要潜热、湿空气焓及水分相态等资料。对于本题给定的数据，本文将预测限定在上述经验温湿方程所描述的范围內。

6 问题三：固定半径药材的达标时间

6.1 全域判据与事件定位

设 $C_{\lim} = 0.15 \text{ kg/kg}$ 。连续模型的干燥时间定义为

$$t_* = \inf\{t > 0 : \max_{0 \leq r \leq R_0} C(r, t) < C_{\lim}\}. \quad (17)$$

对连续变化的轨迹，首次达到阈值与严格低于阈值的时间下确界相同。数值实现先检测相邻积分步的 $\max_i C_i$ 是否跨过 0.15，再在该步内用相同边界插值和后向 Euler 短步进行二分，选择已经满足 $\max_i C_i \leq 0.15$ 的一侧作为输出。最后的全精度状态低于阈值，因此并未把仍然超标的舍入值当作达标。

采用最大值而不是平均值是题目“各处”要求的直接结果。在本文所有主轨迹中，最大值位于中心，但实现仍检查所有节点，避免把中心最湿作为未经检验的程序前提。局部温湿分布或其他边界条件改变时，最大值位置可能需要重新判断。

6.2 达标时间与全程分布

采用附录 3 物性、固定半径、0 至 4 h 观测插值以及 4 h 后末点保持，得到

$$t_{*,3} = 57.1726 \text{ h} \approx 57.17 \text{ h}. \quad (18)$$

全精度定位时刻为 205821.3125 s，最终最大含水率为 0.1499999848 kg/kg。表 5 列出每 6 h 的含水率及末次状态。54 h 时中心仍为 0.1535，尽管表面已降至 0.0528，仍需继续干燥约 3.17 h。若根据表面先达标就停止，会误判内部状态。

表 5 固定半径药材烘干过程的干基含水率（单位：kg/kg）

时间 / h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm
6	1.0156	0.9868	0.9006	0.7546	0.5335
12	0.4548	0.4418	0.4020	0.3289	0.1640
18	0.2979	0.2906	0.2679	0.2247	0.0843
24	0.2371	0.2321	0.2161	0.1851	0.0663
30	0.2053	0.2013	0.1887	0.1637	0.0597
36	0.1854	0.1821	0.1714	0.1501	0.0565
42	0.1717	0.1687	0.1593	0.1404	0.0547
48	0.1615	0.1588	0.1504	0.1332	0.0536
54	0.1535	0.1511	0.1433	0.1275	0.0528
57.1726	0.1500	0.1477	0.1402	0.1249	0.0525

图 5 同时给出最大、干质量加权平均和侧表面含水率，以及代表时刻的径向剖面。平均量的定义为

$$\overline{C}(t) = \frac{2}{R_0^2} \int_0^{R_0} C(r, t) r \, dr \approx 2 \sum_i V_i C_i(t). \quad (19)$$

图中的平均量先于最大量跨过阈值，说明即使整体失水已较充分，中心仍可能决定最后的干燥时长。越接近终点，最大含水率曲线越平缓，少量剩余水分的迁移需要较长时间，不能用初期斜率线性外推。

完整输出写入 **result3.xlsx**，按 60 s 保存每 0.1 cm 位置的含水率，并额外保存实际终点。内部时间步长为 0.5 s，因此不存在以 60 s 为单位向上取整干燥时间的问题。显示的四位小数用于满足题目制表要求，式(18)保留的有效精度仍应结合后期边界假设理解。

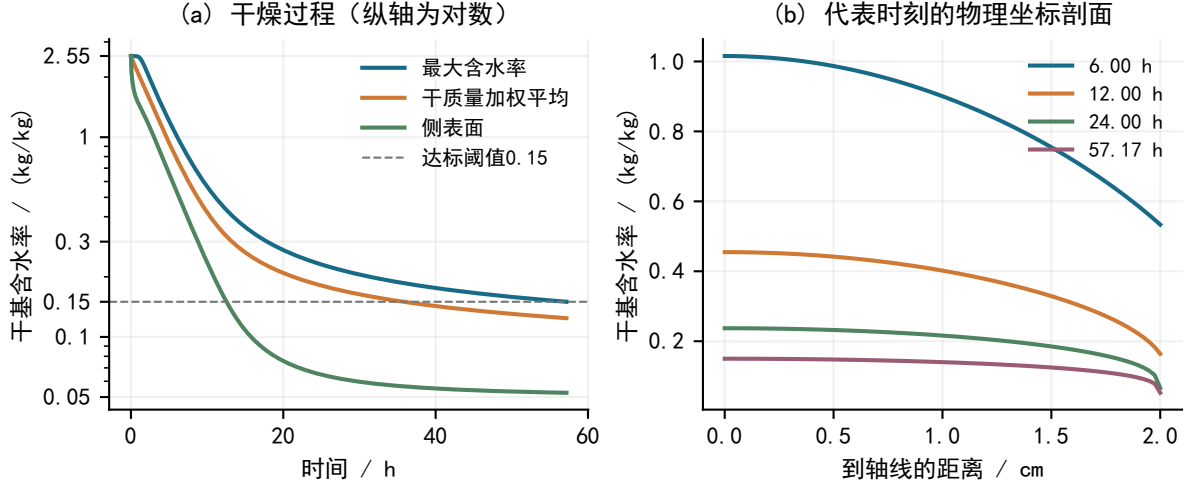


图5 问题三固定半径模型的干燥过程与代表性剖面。主结果采用4 h后末点保持；图中平均值基于均匀独立干固体密度。

7 问题四：规定收缩下的材料坐标模型

7.1 材料运动与干固体守恒

令 ξ 为材料标签，并规定映射

$$r(\xi, t) = R(t)\xi, \quad 0 \leq \xi \leq 1. \quad (20)$$

固定轴向长度时，该映射给出径向速度 $v = (\dot{R}/R)r$ 。一个材料环带的当前体积相对初始体积按 $R(t)^2/R_0^2$ 变化。干固体守恒要求

$$\frac{\partial s}{\partial t} + \frac{1}{r} \frac{\partial (rs v)}{\partial r} = 0. \quad (21)$$

对空间均匀的 $s(t)$ ，代入上述速度得 $\dot{s} + 2(\dot{R}/R)s = 0$ ，从而

$$s(t) = s_0 \left(\frac{R_0}{R(t)} \right)^2, \quad M_d = \pi L s(t) R(t)^2 = \pi L s_0 R_0^2. \quad (22)$$

这里无需指定 s_0 的具体数值即可计算干基含水率及其达标时间，因为后续质量方程的 s_0 会约去。若要报告绝对排水质量，才需要独立测量初始干固体质量，而不能从有效热学密度擅自推算。

干燥收缩可能伴随孔隙、局部密度和结构变化，已有收缩模型的综述也将不同材料和机制下的体积变化关系加以区分^[4]。附件2只提供外形半径轨迹，并不足以识别完整的局部力学与孔隙演化模型。本文据此采用规定形变的工程近似，而不把均匀径向收缩表述为唯一可能的微观机制。

7.2 从真实水质量到干基含水率方程

单位当前体积内的水质量为 sC 。水相对于固体的扩散质量通量取为 $j = -sD C_r$ 。由局部水质量收支得

$$\frac{\partial(sC)}{\partial t} + \frac{1}{r} \frac{\partial}{\partial r} [r(sCv + j)] = 0. \quad (23)$$

将式(21)乘以 C ，再从展开后的式(23)中消去相应项，得到

$$s(C_t + vC_r) = \frac{1}{r} \frac{\partial}{\partial r} (rsDC_r). \quad (24)$$

在均匀形变假设下， s 只随时间变化，因而可以从右侧空间微分中提出并约去。又由于固定 ξ 的网格点与固体一起运动，网格速度 $w = v$ ，有

$$\left. \frac{\partial C}{\partial t} \right|_{\xi} = \left. \frac{\partial C}{\partial t} \right|_r + vC_r. \quad (25)$$

于是问题四的含水率控制方程为

$$\boxed{\left. \frac{\partial C}{\partial t} \right|_{\xi} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left(\xi D(C, \Theta) \frac{\partial C}{\partial \xi} \right)}. \quad (26)$$

式中没有附加的收缩对流项，其原因是材料相对网格的速度为零，而不是为了让预算通过而任意删除一项。这一推导依赖 $v = w$ 、固定长度和均匀干密度三个条件；如果采用非均匀收缩或不同的网格运动，需要重新保留相对网格输运以及空间变化的 s 。

温度采用同一材料导数及有效体积热容，写成

$$a(C) \left. \frac{\partial T}{\partial t} \right|_{\xi} = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} \left(\xi k(C) \frac{\partial T}{\partial \xi} \right). \quad (27)$$

这一温度方程与水分方程在运动描述上相同，但热量闭合仍为前述有效导热近似。两式均使用附录4物性，并在 $\xi = 0$ 施加零梯度，在 $\xi = 1$ 施加

$$-\frac{k}{R} T_{\xi} = h_T(T_s - T_{\text{air}}), \quad -\frac{D}{R} C_{\xi} = h_m(C_s - C_{\text{eq}}). \quad (28)$$

侧面真实水质量通量为 $sh_m(C_s - C_{\text{eq}})$ 。乘以侧面积 $2\pi RL$ 后，与总体水质量的变化一致。因此 s 虽从局部含水率方程中约去，在真实质量解释中仍不可省略。

定义材料参考积分 $I(t) = \int_0^1 C(\xi, t) \xi d\xi$ ，由式(26)及边界条件有

$$\frac{dI}{dt} = -\frac{h_m}{R(t)} (C_s - C_{\text{eq}}), \quad \frac{M_w}{M_d} = 2I(t). \quad (29)$$

式(29)给出了一个直接可计算的质量预算，并与式(16)对应。表格与图中的平均含水率始终采用这一分母，不使用单纯的几何积分 $R(t)^2 I(t)$ 代替干基水质量。

7.3 有效密度与质量定义的一致性

如果将附录 4 的 $\rho(C) = 760 + 90C$ 同时解释为真实的局部湿药材总体密度，那么干基定义会强制给出 $s_\rho(C) = \rho(C)/(1 + C)$ 。固定长度、规定半径和这一密度解释所推算的总干质量为

$$M_{d,\rho}(t) = 2\pi LR(t)^2 \int_0^1 \frac{760 + 90C(\xi, t)}{1 + C(\xi, t)} \xi d\xi. \quad (30)$$

将本文主轨迹代入式(30)，所得到的 $M_{d,\rho}/M_{d,\rho}(0)$ 在保存的时间点上最低为 0.7196，发生于约 7.22 h，终点为 0.8903，见图6。这意味着该额外解释并不满足干固体质量恒定，偏差也远大于离散预算的舍入误差。

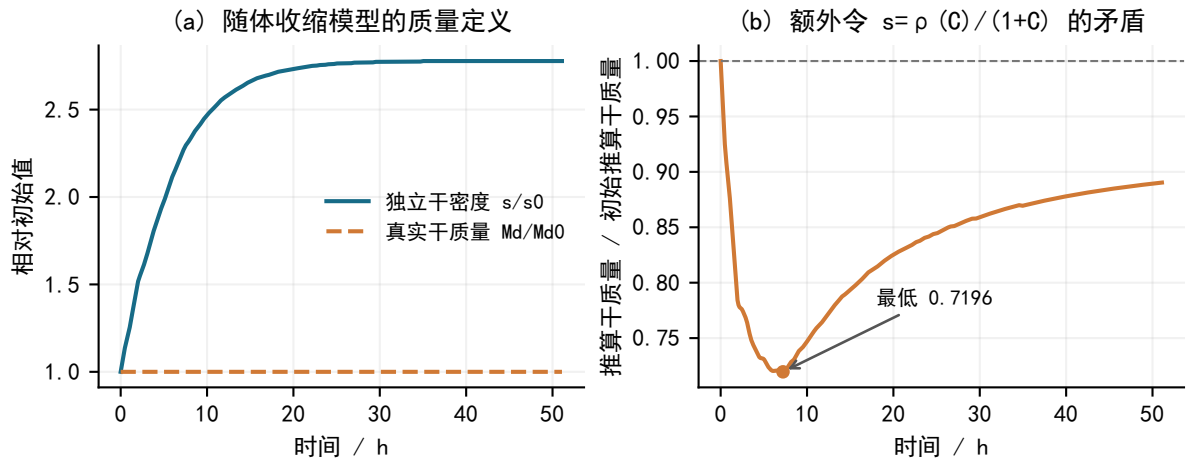


图 6 问题四的两种密度解释。左图为独立干密度保证的质量守恒；右图检验额外把热学密度等同于真实湿密度时的推算干质量。

因此本文保留题目的经验物性数值，但限定其热学用途，以式(22)独立描述守恒干固体。这样得到的是条件清楚、内部一致的有效模型。它没有证明题目的密度公式在真实材料中的含义只能如此解释；若要求 ρ 同时具有严格的湿料质量意义，则必须补充并修改形变、轴向长度或多相结构关系，不能继续保持当前所有假设不变。

7.4 主结果与收缩域上的输出

用均值平台求解式(26)至式(28)，得到

$$t_{*,4} = 51.0913 \text{ h} \approx 51.09 \text{ h}. \quad (31)$$

全精度终点为 183928.625 s，最大含水率 0.1499999894 kg/kg，半径为 1.2000 cm，侧表面含水率按四位小数显示为 0.0526。表6给出每 6 h 的实际位置输出及终点。6 h 时半径已为 1.3740 cm，故 1.5 cm、2 cm 处已经不在药材内，相应位置用横线表示；Excel 中保留为空值，不能填零或把表面值外延到不存在的材料区域。

输出表中 $r = 0.5 \text{ cm}$ 等列对应固定的物理空间距离，不是固定材料标签。对每个输出时刻，先计算当前半径，再以 $\xi = r/R(t)$ 从材料网格插值；若 $r > R(t)$ 则不输出材料

表 6 随体收缩药材烘干过程的干基含水率（单位：kg/kg）

时间 / h	0 cm	0.5 cm	1 cm	1.5 cm	2 cm	侧表面	半径 / cm
6	1.7188	1.5370	1.0221	—	—	0.4205	1.3740
12	0.7374	0.6544	0.4082	—	—	0.1669	1.2480
18	0.4087	0.3686	0.2397	—	—	0.0892	1.2140
24	0.2851	0.2610	0.1790	—	—	0.0673	1.2040
30	0.2263	0.2094	0.1492	—	—	0.0595	1.2010
36	0.1928	0.1796	0.1316	—	—	0.0559	1.2000
42	0.1712	0.1604	0.1200	—	—	0.0541	1.2000
48	0.1561	0.1468	0.1116	—	—	0.0530	1.2000
51.0913	0.1500	0.1413	0.1081	—	—	0.0526	1.2000

数值。侧表面单列直接取 $\xi = 1$ ，因此即使半径不恰好落在 0.1 cm 的采样网格上，也能保留准确的表面状态。**result4.xlsx** 按 60 s 输出并额外保留终点，避免收缩后表格位置含义不清。

图7展示主模型的全程下降与代表性剖面。后期曲线仍存在明显尾段，中心是达标约束的控制位置。终点平均含水率约为 0.11835，低于中心的阈值 0.15，说明达标时内部并非处处具有相同含水率。只要所有位置低于上限，保留这种空间差异并不违反题目要求。

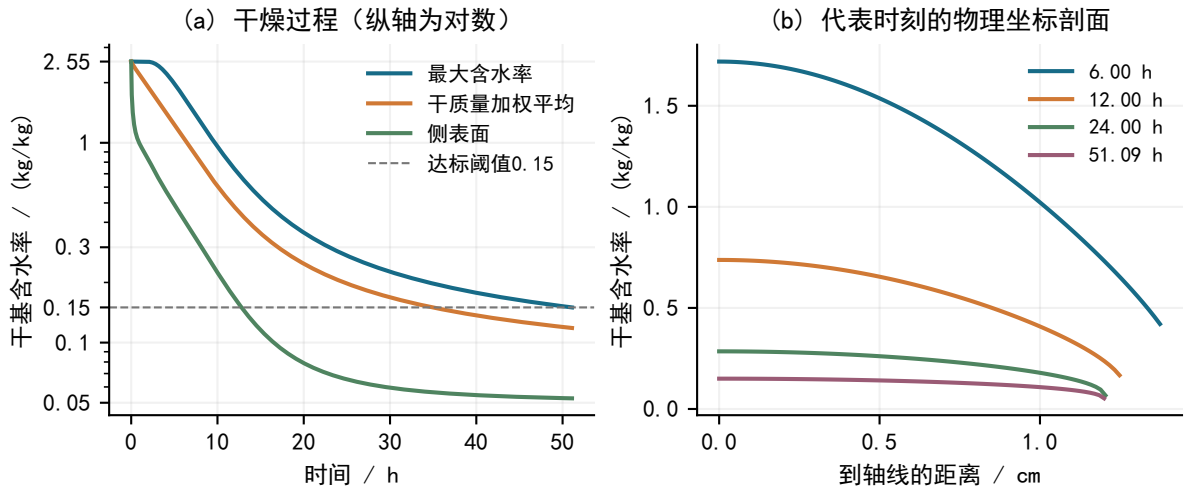


图 7 问题四材料坐标、末小时均值平台下的干燥结果。右图横轴为当前物理距离，各条曲线的终点随半径改变。

8 模型对照、数值验证与敏感性分析

8.1 同物性、同边界的严格对照

为了单独考察几何和运动描述，三组对照均使用附录 4 物性、均值平台、81 节点、0.5 s 步长及相同初始状态。第一组固定 $R = R_0$ ，第二组采用本文随体收缩，第三组将移动边界应用在没有固体径向运动的 Eulerian 扩散场上。这第三组属于不同的运动假设，保留它是为了说明坐标变换和材料守恒的区别，不将其视为本文主解。

对于没有物质速度的 Eulerian 扩散方程，单纯变换 $r = R(t)\xi$ 会出现

$$C_t|_\xi = \frac{1}{R^2\xi} \partial_\xi(\xi DC_\xi) + \frac{\dot{R}}{R} \xi C_\xi. \quad (32)$$

链式法则中的符号本身没有错误，但它与式(26)描述的物理运动不同。此时几何积分的预算包含 $R\dot{R}C_s$ ，即边界扫掠项：

$$\frac{d}{dt}(R^2I) = -Rh_m(C_s - C_{eq}) + R\dot{R}C_s. \quad (33)$$

所以通过几何积分预算，并不能证明以守恒于固体为分母的水质量正确。把“离散守恒”与“守恒的物理量是什么”同时写清，才能判断两种运动描述的适用性。

表7与图8给出完整对照。相同物性和平台下，固定半径需要 129.8561 h，随体收缩需要 51.0913 h，缩短 78.7649 h，约为 60.66%；Eulerian 移动对照为 52.6548 h，比材料坐标结果增加 1.5636 h。可见，固定半径与移动域的巨大差异主要出现在几何路径改变后，而额外扫掠输运只解释两种移动描述之间的一部分差别。

表 7 问题四同物性、同均值平台的运动与几何对照

计算情景	初始半径/cm	终点半径/cm	达标时间/h
固定半径	2.0000	2.0000	129.8561
随体归一化坐标	2.0000	1.2000	51.0913
Eulerian 移动边界对照	2.0000	1.2000	52.6548

收缩使水分迁移距离缩短，同时提高单位干质量对应的侧面交换作用。半径由 0.02 m 降至 0.012 m 后，在归一化方程中 $1/R^2$ 增加为初始的约 2.778 倍， h_m/R 增加为约 1.667 倍。这一尺度变化可以解释缩短方向，但物性随温湿场变化，且半径并非瞬间缩至终值，故不能直接用 0.6^2 乘固定半径时长作为精确答案。

8.2 边界延续情景对结果的影响

材料坐标下，末点保持得到 50.8235 h，末小时均值平台得到 51.0913 h，名义平台得到 51.0899 h。末小时均值与名义平台仅相差约 5.13 s，而末点保持比均值平台提前约

16.07 min，结果见图8左图。由于末点的温度略高、水分变量略低，其延续条件更有利于干燥，时间缩短的方向与方程一致。

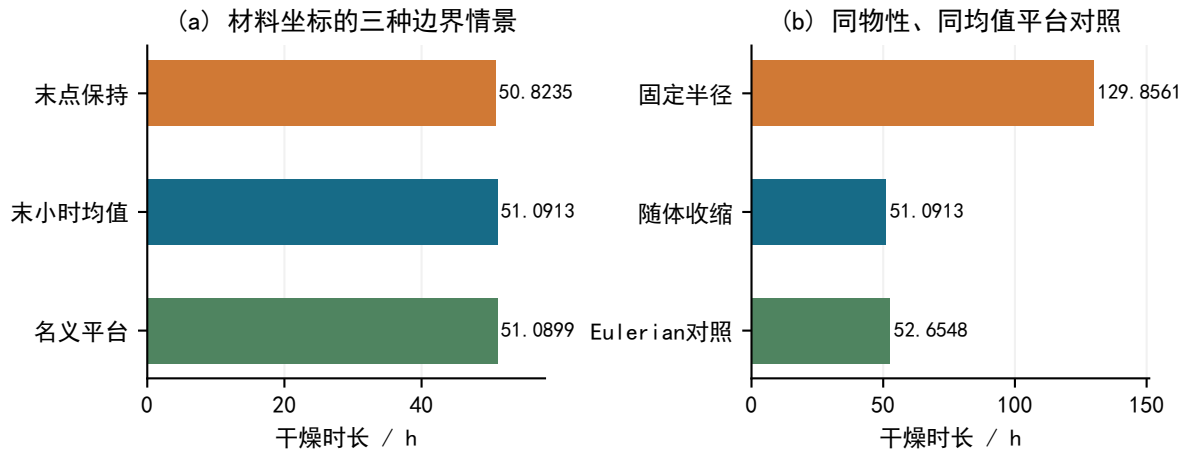


图8 问题四的边界情景与严格对照。左图均为材料坐标；右图统一采用附录4物性和末小时均值平台。

三种相邻平台情景得到约 50.82 至 51.09 h 的范围，只是指定的边界延续比较，不是统计置信区间，也不覆盖未给出的全部工艺可能。若实际后期升温、降温、间歇通风或水分平台改变，结果需要重新计算。本文选择均值平台的依据是利用后段多个观测值形成代表性稳定水平，而不是已知未来边界必然恒定。

问题三和问题四的主时长分别为 57.17 h 和 51.09 h，虽然数值相近，不能据此说收缩仅带来约 6 h 的收益。两问使用的扩散系数前因子及含水率指数不同，热物性也不同，边界延续还存在差别。表7的配对比较才是在指定附录4物性下讨论收缩影响的依据。不同对照所回答的问题不同，合并解释会掩盖参数变化与运动学变化的各自作用。

8.3 求解器与守恒验证

先以可独立构造的问题检查离散实现，再考察非线性主模型的收敛。Thomas 算法与独立带状矩阵求解比较，最大差约为 1.07×10^{-14} 。对移动几何下的常数场，分别检查收缩和膨胀、Euler 和 BDF2 组合，最大误差约为 4.34×10^{-10} ，用于检验几何离散的一致性。该常数场试验检验的是离散几何恒等式，不能替代第7.3节的物理质量解释。

圆柱径向离散的独立验证采用 Robin 特征模态。在常系数、零外界扰动的情況下，取

$$u(r, t) = J_0(\lambda r/R_0) \exp(-\alpha \lambda^2 t/R_0^2), \quad \lambda J_1(\lambda) = \text{Bi}_T J_0(\lambda), \quad (34)$$

其中 J_0, J_1 为第一类 Bessel 函数。使用解析初值，时间步长 0.05 s，积分到 100 s，与解析场比较。表8列出结果：节点由 21 增至 41、81 时，最大场误差持续下降，支持轴线、径向控制体和 Robin 表面项的正确离散。

非线性界面求积的验证沿三个不同含水率区间和 45 至 50 °C 的重构温度路径进行，

表 8 独立数值验证结果

验证对象与误差定义	误差
Thomas 与独立带状求解器：最大绝对差	1.0658×10^{-14}
移动几何常数场：最大绝对差	4.3396×10^{-10}
Robin 模态，21 节点：最大绝对场误差	9.1433×10^{-5}
Robin 模态，41 节点：最大绝对场误差	2.2193×10^{-5}
Robin 模态，81 节点：最大绝对场误差	4.8579×10^{-6}
8 点界面求积与自适应积分：最大相对差	3.6731×10^{-9}
材料坐标 Euler 独立算例：累计相对水分残差	3.0477×10^{-15}
材料坐标 BDF2 独立算例：累计相对水分残差	1.4368×10^{-14}

分别用自适应数值积分和 8 点 Gauss 规则计算式(13)，最大相对差为 3.67×10^{-9} 。它针对的是界面系数积分，而不是对整个药材模型的外部实验拟合。

问题四主轨迹的材料参考积分累计相对残差约为 4.67×10^{-14} ，与离散预算一致。该主轨迹统计覆盖内部整步，不包含最后事件二分产生的短步；二维端面对照另有包含末次短步的完整预算。报告统计范围，可以避免将一个未纳入累计量的子步骤误称为已经验证。所有这些残差均衡量代数实现和数值收支，不表示物理预测误差已经达到机器精度。

8.4 时间、空间和插值收敛

对问题四主模型，所有收敛算例统一使用材料坐标和均值平台。空间试验固定 $\Delta t = 1 \text{ s}$ ，分别采用 41、81、161 节点；时间试验固定 81 节点，使用 4、2、1、0.5 s；另计算 BDF2 和线性边界插值的情景。表9和图9给出实际达标时间。

表 9 问题四当前材料模型的离散与插值比较

比较项	节点数	步长/s	方法与插值	达标时间/h
空间粗级	41	1	Euler, PCHIP	51.0988021
空间中级	81	1	Euler, PCHIP	51.0916146
空间细级	161	1	Euler, PCHIP	51.0897917
时间粗级	81	4	Euler, PCHIP	51.0935938
时间中级	81	2	Euler, PCHIP	51.0922743
主结果	81	0.5	Euler, PCHIP	51.0912847
时间格式	81	1	BDF2, PCHIP	51.0909549
边界插值	81	1	Euler, 线性	51.0916146

固定 1 s 时，81 节点加密到 161 节点，时间改变 0.0018229 h，即 6.5625 s；固定 81 节点时，1 s 减至 0.5 s，时间改变 1.1875 s。Euler 与 BDF2 在 1 s 下相差 2.3750 s。线性插值与 PCHIP 在该次试验中给出相同的事件输出，说明在当前时刻定位分辨率下没有分辨出两者对达标时间的差异；这并不表示任意时刻的完整温湿场都严格相同。

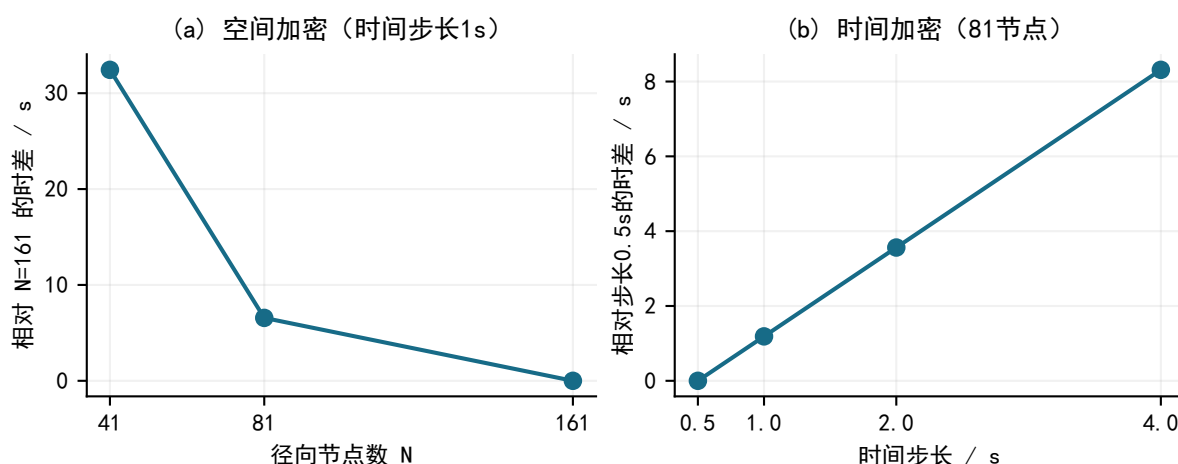


图 9 问题四材料模型的空间与时间加密。纵轴分别相对于最细空间和最细时间算例，不能作为连续解误差的严格上界。

这些结果支持 51.09 h 不是明显的离散偶然值。主轨迹的最终事件括区宽度为 0.0625 s，但这一宽度只描述给定离散轨迹上的定位分辨率，不能作为总预测精度。特别是，均值与末点边界导致的约 16 min 差异远大于此次加密带来的秒级变化，因此继续显著缩小时间步长，并不能消除后期工艺边界的信息缺口。

8.5 参数敏感性

为考察经验参数的不确定影响，分别给 D, h_m, h_T, k 乘以 0.8、0.9、1.0、1.1 和 1.2，其余条件保持不变。每次重新求解完整非线性轨迹，统一使用 81 节点、1 s、材料坐标和均值平台。基准时长为 51.0916146 h，稍异于 0.5 s 的主结果，故所有百分比均相对于本节共同基准计算。 D 的倍率作用于整个经验函数，不改变其对 C, Θ 的函数形式；参数变化时半径轨迹仍按附件 2 规定。

表10和图10显示，扩散系数对时长的影响最大。 D 减小 20% 时，达标时间增至 62.3506 h，增加 22.04%； D 增大 20% 时降至 43.6211 h，减少 14.62%。响应并非对称的线性关系，这与扩散系数随状态变化及达标阈值位于后期减速段有关。

对 h_m ，减小 20% 延长约 1.34%，增大 20% 缩短约 0.77%，明显弱于同幅度的 D 变化。 h_T 和 k 的 20% 变化对全程时长的影响更小，最大相对变化分别约 0.0355% 与 0.0087%。这与药材在初期已接近热平台、后期主要受低含水率扩散制约的结果相符。这里的结论针对达标时间，不等于热参数对早期温度场也同样不敏感。

指定倍率试验没有给参数建立概率分布，因此表10不产生置信区间。由于缺少真实

表 10 问题四单参数倍率试验的达标时间（单位：h）

参数	$\times 0.8$	$\times 0.9$	$\times 1.0$	$\times 1.1$	$\times 1.2$
D	62.3506	56.0918	51.0916	47.0116	43.6211
h_m	51.7773	51.3827	51.0916	50.8708	50.6994
h_T	51.1098	51.0997	51.0916	51.0851	51.0797
k	51.0960	51.0936	51.0916	51.0901	51.0888

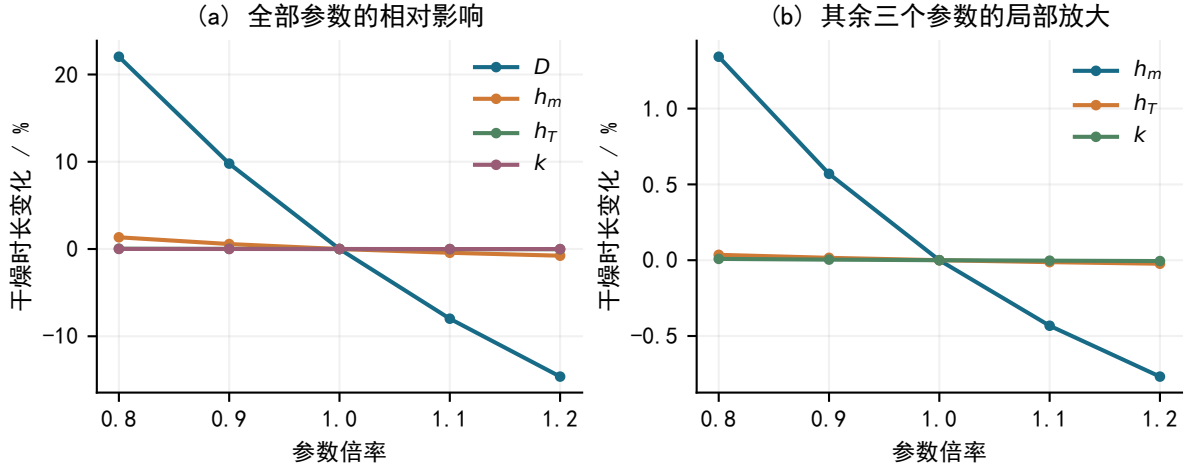


图 10 问题四参数倍率对达标时间的影响。右图将其余参数的响应单独放大；两图百分比采用同一 1 s 基准轨迹。

参数分布及 4 h 后工艺记录，本文不将大量随机抽样的分位数包装成经过数据支持的预测概率。若进一步开展不确定性量化，应先确定扩散系数和边界延续的可用信息，再决定抽样规模，而不能以增加计算量代替物理输入。

9 轴对称端面对照与适用范围

9.1 二维模型与配对设计

为检验一维近似，对问题四主模型增加轴向坐标 z ，利用中截面对称只计算 $0 \leq z \leq H = 0.125$ m。在相同随体径向形变下，水分方程为

$$C_t = \frac{1}{R(t)^2 \xi} \partial_\xi (\xi D C_\xi) + \partial_z (D C_z), \quad (35)$$

温度方程采用同样的空间算子，以 a 乘时间导数、以 k 替换 D 。在 $z = 0$ 施加对称条件；在 $z = H$ 施加与侧面相同的 h_T, h_m 和 C_{eq} 。端面系数相同是一项用于对照的暴露条件，题目并未独立测定端面的传递系数。

径向采用与一维相同的控制体，轴向节点使用 $z = H[1 - (1 - u)^2]$ ，其中 u 均匀取点，以增加端面附近的分辨率。时间上使用全隐式 Euler 与 Picard 迭代，线性子问题采

用稀疏直接求解。关闭端面交换时，二维场应退化为沿轴向重复的一维解；开放端面后，再与相同径向网格和时间步长的一维解配对，避免将离散设置变化当成端面作用。

令轴向控制体宽度为 W_j ，参考半域体积权重为 $V_{\text{ref}} = \sum_{ij} V_i W_j = H/2$ ，则

$$\begin{aligned} \bar{C} &= \frac{\sum_{ij} V_i W_j C_{ij}}{V_{\text{ref}}}, \\ F_s &= \frac{h_m}{RV_{\text{ref}}} \sum_j W_j (C_{s,j} - C_{\text{eq}}), \quad F_e = \frac{h_m}{V_{\text{ref}}} \sum_i V_i (C_{i,H} - C_{\text{eq}}). \end{aligned} \quad (36)$$

其中 F_s, F_e 分别为相对于总干质量的侧面和端面失水速率。完整圆柱的周向系数、两个端面对称半域倍数已在分子分母中约去，故不会因为只计算一半长度而漏掉一个端面。二维预算逐步检查 $\bar{C}^n - \bar{C}^{n-1} + \Delta t(F_s + F_e) = 0$ ，并包含最后事件短步。

9.2 结果与解释

表11列出闭端退化试验及两级开放端面对照。闭端计算的二维场与重复的一维场在代表时刻的最大温差约为 $1.95 \times 10^{-12} \text{ K}$ ，最大含水率差约为 $4.49 \times 10^{-14} \text{ kg/kg}$ ，说明该实现能够回到一维模型。

表 11 端面试验与同设置一维解的配对结果

端面设置	径向 × 轴向节点	步长/s	一维时间/h	二维时间/h
关闭交换	41 × 9	30	51.1178874	51.1178874
开放粗级	41 × 25	30	51.1178874	51.1178874
开放细级	81 × 37	15	51.1008301	51.1008301

两级开放计算在当前事件定位分辨率内都没有分辨出配对达标时差。细级二维全程相对水分预算残差约为 5.61×10^{-14} ，累计端面排水占全部排水的 3.9032%。细级配对时间 51.1008301 h 比 0.5 s 主结果略大，是较粗时间设置下的数值结果；不能把这一绝对差直接当作端面误差。

另一方面，二维开放端面确实改变了平均含水率。表12与图11显示，4 h 时一维平均值为 1.42299055，二维降至 1.37869807，变化约 -3.1126%；终点一维为 0.11834815，二维为 0.11607800，变化约 -1.9182%。因此，“未分辨出全域最大值达标时差”与“平均量和端部场有变化”可以同时成立。

中截面到端面的距离 0.125 m 大于径向扩散距离 0.02 至 0.012 m，端部失水可先改变端部附近的局部场，而控制最迟达标的中心区域仍主要通过径向排水。这一尺度解释与本组对照相符，但本文不把重合的离散事件括区解释为连续偏微分方程的严格误差上界。不同端面传递系数、不同长径比或存在轴向收缩时，需要重新检验。

表 12 细级端面对照在相同时刻的平均含水率

时间/h	一维平均 C	二维平均 C	相对变化/%
0.5000	2.34383354	2.32874268	-0.6439
4.0000	1.42299055	1.37869807	-3.1126
24.0000	0.20541408	0.19931230	-2.9705
51.1008	0.11834815	0.11607800	-1.9182

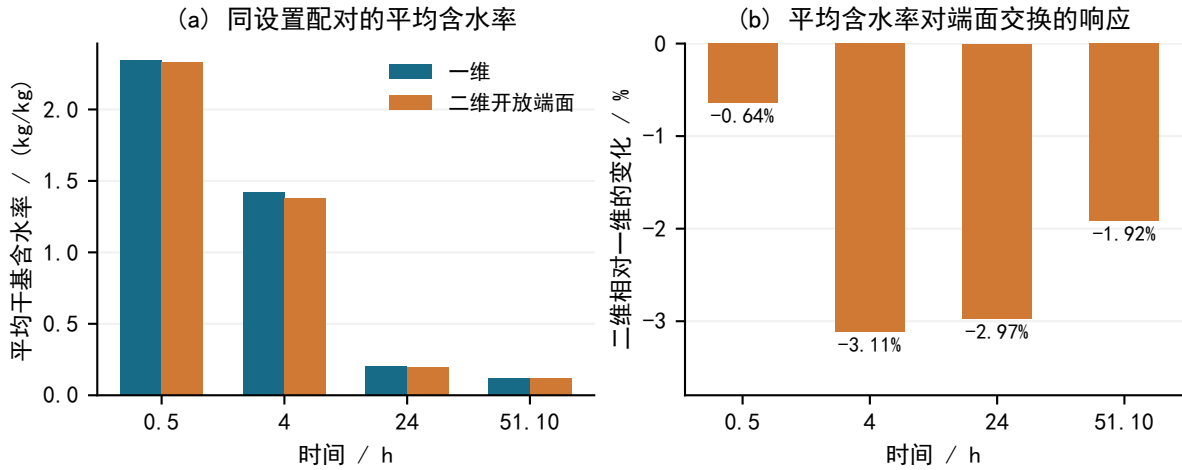


图 11 同网格、同时间步长下，端面交换对平均含水率的影响。此处比较量为平均含水率，而不是全域最大含水率的达标时间。

10 模型评价与结论

10.1 有效性与局限

本文以相同控制体结构求解四问，使几何、物性和边界的改变可以分开追踪。圆柱权重避免了中心奇点，隐式非线性迭代能够处理长达数十小时的低含水率尾段；解析模态、独立线性求解、界面求积和时间空间加密分别检验不同数值环节。问题四从干固体连续方程出发定义水质量，明确了材料坐标中无相对网格输运的条件，并以同物性对照解释固定与收缩半径的时间差。

模型的限制主要来自可用物理信息。第一，4 h 后边界延续是情景，尚无后续工艺观测。第二，空气变量到等效表面含水率的映射缺少吸附数据，传质边界属于经验闭合。第三，半径仅由附件规定，均匀径向形变和热学有效密度解释需要独立实验支持；参数扫描中也没有重新求解含水率影响收缩的力学反馈。第四，温度方程未描述潜热、多相焓和结构损伤，不能用于直接推断能耗或品质变化。第五，端面对照只验证了特定暴露条件，不能推广为所有局部场的端面作用均可忽略。

上述不确定性对不同输出量的影响并不相同。端面对最大值达标时间和平均含水率

的不同作用已经说明，不能把所有误差来源排成一条适用于任何指标的固定大小顺序。对于本题的时长预测，当前网格与时间步比较已经达到秒级差异；对于平均失水量、端部状态或能耗，则需要针对相应物理量补充数据和验证。

10.2 四问结论

问题一得到表1、表2及图3所示结果。前 30 min 热量向中心传播明显快于水分变化：1800 s 时中心与表面温度分别为 33.5764 °C 和 36.7861 °C，表面含水率降至 1.5104，中心仍接近 2.55。问题二在全程使用附录 3 后，3 h 时中心温度升至 49.8494 °C，但最大含水率仍为 1.7662，说明达到近恒温状态远早于完成干燥。

问题三在固定半径、附录 3 物性和末点延续条件下，首次全域达标时间为 57.1726 h，完整水分剖面见表5。问题四在规定半径、均匀随体径向收缩、独立于固体守恒、经验有效热物性及末小时均值平台下，推荐时间为 51.0913 h，结果见表6和图7。末点保持与名义平台分别得到 50.8235 h 和 51.0899 h，作为并列边界情景保留。

严格配对计算表明，同附录 4 物性和均值平台下，固定半径需要 129.8561 h，收缩显著缩短迁移距离，材料坐标解为 51.0913 h。Eulerian 扫掠边界对照为 52.6548 h，其守恒对象与材料水质量不同，不能替代主模型。参数试验中扩散系数对时长最敏感；端面对照支持在本文条件下用径向模型预测最迟达标时刻，同时提示平均含水率仍受端面交换影响。以上结论连同四份 Excel，构成给定资料与明确假设下的完整计算回答。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型推导讨论、物理假设审查、数值程序编写与调试、结果核对、文献检索和论文初稿撰写，详细使用情况见支撑材料。

参考文献

- [1] DONEA J, HUERTA A, PONTHOT J P, et al. Arbitrary Lagrangian–Eulerian methods [M/OL]//STEIN E, DE BORST R, HUGHES T J R. Encyclopedia of Computational Mechanics: Vol. 1. John Wiley & Sons, 2004: 413-437. DOI: 10.1002/0470091355.ecm009.
- [2] FRITSCH F N, BUTLAND J. A method for constructing local monotone piecewise cubic interpolants[J/OL]. SIAM Journal on Scientific and Statistical Computing, 1984, 5(2): 300-304. DOI: 10.1137/0905021.
- [3] EYMARD R, GALLOUËT T, HERBIN R. Finite volume methods[M/OL]//CIARLET P G, LIONS J L. Handbook of Numerical Analysis: Vol. 7. Elsevier, 2000: 713-1018. DOI: 10.1016/S1570-8659(00)07005-8.
- [4] MAYOR L, SERENO A M. Modelling shrinkage during convective drying of food materials: A review[J/OL]. Journal of Food Engineering, 2004, 61(3): 373-386. DOI: 10.1016/S0260-8774(03)00144-4.

附录 A 支撑材料及运行说明

表13列出与本文结果对应的文件。程序按模块组织，所有计算使用相对路径读取同一目录内的数据；原始 Excel 可与转换后的 CSV 逐项核对。以下程序清单给出正文涉及的求解、验证、对照和导出代码。源码中的模型编号 1、2、4 分别表示问题一、问题二和三、问题四；编号差异没有物理量纲含义。

表 13 支撑材料文件列表

文件或目录	内容与用途
result1.xlsx、result2.xlsx	温度和水分浓度, 1 s 时间间隔、0.1 cm 空间间隔
result3.xlsx、result4.xlsx	水分浓度, 60 s 时间间隔并包含终点; 问题四保留独立表面列
data/boundary.csv、data/radius.csv	附件 1 与附件 2 的数值转存, 保持原始时刻与单位
run.py、physics/、solver/	四问主计算、物性、外部边界和一维有限体积求解器
experiments/	问题四三种平台、几何运动对照、收敛与单参数扫描
validation/、tests/	二维端面对照、输出一致性检查及独立数值测试
export/	从全精度结果形成题目指定工作表
results/q1.npz 至 q4.npz	按规定输出频率保存的全精度径向轨迹
results/q1.json 至 q4.json	物性选择、网格步长、边界和达标状态
results/中的比较 CSV 与 JSON	本文收敛、参数与端面比较的数值记录
paper/prepare_paper_assets.py	从现有数值轨迹生成本文图与六张题目结果表
AI 工具使用详情.pdf	已知 AI 参与环节、提示方式及核验情况说明

主计算依赖 Python、NumPy、SciPy 和 Numba; 绘图依赖 Matplotlib; 现有 Excel 导出程序依赖 Node.js 及 @oai/artifact-tool。本文数值结果使用 Python 3.10、NumPy 1.26.4、SciPy 1.15.2 及 Numba 0.67.0 生成。二维计算比一维更耗时, 但只需执行本文给

出的三个配对算例，不要求多卡设备。

在程序根目录执行 `python run.py` 可依次计算四问；执行实验目录中的问题四对照与参数脚本可生成本文相应表格。二维脚本的三个设置依次为径向 41、轴向 9、30 s 且关闭端面；径向 41、轴向 25、30 s 且开放端面；径向 81、轴向 37、15 s 且开放端面，均启用同设置一维配对。主结果的半径列以 m 存储，Excel 输出时转换为 cm；时间轨迹统一用 s 存储，正文表格按题目要求转换为 h。

附录 B 完整源程序

B.1 主计算入口

文件：run.py。

Listing 1 主计算入口

```
1 """Full calculation. Run from any directory: python A_model/run.py."""
2 from pathlib import Path
3 import argparse
4 import json
5 import time
6 import numpy as np
7 from solver.fvm_cpu import solve
8 from physics.material import MODEL_Q1, MODEL_Q23, MODEL_Q4
9
10 ROOT = Path(__file__).resolve().parent
11
12 def main():
13     parser=argparse.ArgumentParser()
14     parser.add_argument('--n',type=int,default=81)
15     parser.add_argument('--dt',type=float,default=.5)
16     parser.add_argument('--questions', type=int, nargs='+', choices=range(1,5), default=[1,2,3,4])
17     args=parser.parse_args()
18     outdir=ROOT/'results'; outdir.mkdir(exist_ok=True)
19     for q in args.questions:
20         start=time.perf_counter()
21         out,meta=solve(model=MODEL_Q1 if q==1 else MODEL_Q4 if q==4 else MODEL_Q23, moving=q==4,n=args.n,dt=
args.dt,
22                         end=1800 if q==1 else 10800 if q==2 else 432000,interval=1 if q<=2 else 60,event=q>=3,
23                         ale=q!=4, tail='mean' if q==4 else 'last')
24         meta['wall_seconds']=time.perf_counter()-start
25         np.savez_compressed(outdir/f'q{q}.npz',data=out)
26         (outdir/f'q{q}.json').write_text(json.dumps(meta,indent=2),encoding='utf-8')
27         print(f'Q{q} ' +json.dumps(meta),flush=True)
28
29 if __name__=='__main__': main()
```

B.2 经验物性与界面扩散系数

文件: physics/material.py。

Listing 2 经验物性与界面扩散系数

```
1 import numpy as np
2 from numba import njit
3
4 MODEL_Q1 = 1
5 MODEL_Q23 = 2
6 MODEL_Q4 = 4
7
8 @njit(cache=True)
9 def validate_model(model):
10     if model != MODEL_Q1 and model != MODEL_Q23 and model != MODEL_Q4:
11         raise ValueError('Unknown model: use MODEL_Q1=1, MODEL_Q23=2, MODEL_Q4=4')
12
13 GX, GW = np.polynomial.legendre.leggauss(8)
14 GX, GW = (GX+1)/2, GW/2
15
16 @njit(cache=True)
17 def moisture_faces(T, C, model, mode, scales):
18     """Kirchhoff/secant diffusivity: integrate D along each reconstructed edge.
19
20     Harmonic averaging of endpoint D badly resolves the exponential dry skin.
21     Positive Gauss quadrature preserves a positive transmissibility.
22     """
23     validate_model(model)
24     result = np.zeros(len(C)-1)
25     for i in range(len(result)):
26         for j in range(len(GX)):
27             c = C[i]*(1-GX[j])+C[i+1]*GX[j]
28             t = T[i]*(1-GX[j])+T[i+1]*GX[j]+273.15
29             if model == MODEL_Q1:
30                 d = 7e-9*np.exp(-.89/c)
31             elif model == MODEL_Q23:
32                 d = 2.4e-3*np.exp(-.45/c-3850/t)
33             if mode == 2:
34                 d = 2.4e-3*np.exp(-.45/2.55-3850/301.15)
35             elif model == MODEL_Q4:
36                 d = 4.2e-4*np.exp(-.30/c-3850/t)
37             result[i] += GW[j]*d*scales[0]
38     return result
39
40 @njit(cache=True)
41 def properties(T, C, model, mode, scales):
42     """SI units; scales ordered D, hm, hT, k. mode 1 freezes thermal properties."""
43     validate_model(model)
44     n = len(C)
45     capacity, k, D = np.empty(n), np.empty(n), np.empty(n)
46     for i in range(n):
47         c = C[i]
48         if c <= 0 or T[i] <= -273.15:
49             raise ValueError('Nonphysical Picard iterate')
50         ct = 2.55 if mode == 1 else c
51         if model == MODEL_Q1:
```

```

52     capacity[i], k[i] = 820.0 * 2600.0, 0.36
53     D[i] = 7e-9 * np.exp(-0.89 / c)
54     elif model == MODEL_Q23:
55         capacity[i] = (650 + 128 * ct) * (1450 + 2736 * ct / (ct + 1))
56         k[i] = 0.21 + 0.38 * ct / (ct + 1)
57         D[i] = 2.4e-3 * np.exp(-0.45 / c - 3850 / (T[i] + 273.15))
58     elif model == MODEL_Q4:
59         capacity[i] = (760 + 90 * ct) * (1850 + 2150 * ct / (ct + 1))
60         k[i] = 0.12 + 0.20 * ct / (ct + 1)
61         D[i] = 4.2e-4 * np.exp(-0.30 / c - 3850 / (T[i] + 273.15))
62     if mode == 2: # Full fixed-parameter ablation, frozen at the initial state.
63         if model == MODEL_Q23:
64             capacity[i] = (650+128*2.55)*(1450+2736*2.55/3.55)
65             k[i] = 0.21+0.38*2.55/3.55
66             D[i] = 2.4e-3*np.exp(-0.45/2.55-3850/301.15)
67         k[i] *= scales[3]
68         D[i] *= scales[0]
69     return capacity, k, D

```

B.3 观测插值和边界延续

文件: physics/boundary.py。

Listing 3 观测插值和边界延续

```

1  from pathlib import Path
2  import numpy as np
3  from scipy.interpolate import PchipInterpolator
4
5  DATA = Path(__file__).resolve().parents[1] / 'data'
6
7  class Boundary:
8      def __init__(self, interpolation='pchip', tail='last'):
9          self.data = np.loadtxt(DATA / 'boundary.csv', delimiter=',', skiprows=1)
10         self.radius_data = np.loadtxt(DATA / 'radius.csv', delimiter=',', skiprows=1)
11         self.interpolation = interpolation
12         self.pchip = PchipInterpolator(self.data[:, 0], self.data[:, 1:], axis=0)
13         self.rpchip = PchipInterpolator(self.radius_data[:, 0], self.radius_data[:, 1] / 100)
14         if tail not in ('last', 'mean', 'nominal'):
15             raise ValueError('tail must be last, mean, or nominal')
16         self.tail = (self.data[-1, 1:] if tail == 'last' else
17                     np.array([50., .05]) if tail == 'nominal' else
18                     self.data[self.data[:, 0] >= 10800, 1:].mean(axis=0))
19
20     def ambient(self, time):
21         t = np.asarray(time)
22         tc = np.clip(t, 0, self.data[-1, 0])
23         if self.interpolation == 'pchip':
24             values = self.pchip(tc)
25         else:
26             values = np.stack([np.interp(tc, self.data[:, 0], self.data[:, j]) for j in (1, 2)], axis=-1)
27         return np.where((t > self.data[-1, 0])[..., None], self.tail, values)
28
29     def radius(self, time):
30         return self.rpchip(np.clip(time, 0, self.radius_data[-1, 0]))

```

B.4 一维有限体积求解器

文件: solver/fvm_cpu.py。

Listing 4 一维有限体积求解器

```
1 """Nodal cylindrical FVM, conservative ALE, implicit Picard and BE/BDF2."""
2 import numpy as np
3 from numba import njit
4 from physics.material import properties, moisture_faces, validate_model, MODEL_Q23, MODEL_Q4
5 from physics.boundary import Boundary
6
7 @njit(cache=True)
8 def geometry(n):
9     faces = np.empty(n+1)
10    faces[0], faces[-1] = 0., 1.
11    for i in range(1, n):
12        faces[i] = (i-0.5)/(n-1)
13    return faces, 0.5*(faces[1:]**2-faces[:-1]**2)
14
15 @njit(cache=True)
16 def assemble(p, capacity, old, prev, R, Ro, Rp, dt, alpha, beta, gamma, h, ambient, ale):
17     n = len(old)
18     faces, v = geometry(n)
19     # Material-coordinate comparison has no grid-advection term.
20     if not ale:
21         Ro, Rp = R, R
22         g = (alpha*R*R-beta*Ro*Ro+gamma*Rp*Rp)/(2*dt)
23         if abs(g) < 1e-20:
24             g = 0.
25         diag = capacity * alpha * R*R * v / dt
26         rhs = capacity * v * (beta*Ro*Ro*old-gamma*Rp*Rp*prev) / dt
27         lower, upper = np.zeros(n-1), np.zeros(n-1)
28         for j in range(n-1):
29             pf = p[j] if len(p) == n-1 else 2*p[j]*p[j+1]/(p[j]+p[j+1])
30             G = faces[j+1]*(n-1)*pf
31             diag[j] += G
32             diag[j+1] += G
33             upper[j] -= G
34             lower[j] -= G
35             # Conservative grid flux; upwind for standard velocity -Rdot*xi/R.
36             q = g * faces[j+1]**2
37             if abs(q)*max(capacity[j],capacity[j+1]) <= 1.8*G:
38                 # Centered grid flux is second order when the face Peclet number
39                 # keeps both off-diagonal coefficients nonpositive.
40                 diag[j] -= .5*capacity[j]*q
41                 upper[j] -= .5*capacity[j]*q
42                 lower[j] += .5*capacity[j+1]*q
43                 diag[j+1] += .5*capacity[j+1]*q
44             elif q <= 0:
45                 diag[j] -= capacity[j]*q
46                 lower[j] += capacity[j+1]*q
47             else:
48                 upper[j] -= capacity[j]*q
49                 diag[j+1] += capacity[j+1]*q
50     diag[-1] += R*h - capacity[-1]*g
51     rhs[-1] += R*h*ambient
```

```

52     return lower, diag, upper, rhs, g
53
54 @njit(cache=True)
55 def thomas(lower, diag, upper, rhs):
56     b, x = diag.copy(), rhs.copy()
57     for i in range(1, len(b)):
58         z = lower[i-1]/b[i-1]
59         b[i] -= z*upper[i-1]
60         x[i] -= z*x[i-1]
61     x[-1] /= b[-1]
62     for i in range(len(b)-2, -1, -1):
63         x[i] = (x[i]-upper[i]*x[i+1])/b[i]
64     return x
65
66 @njit(cache=True)
67 def step(Told, Cold, Tprev, Cprev, R, Ro, Rp, ambient, dt, model, mode, scales, omega, alpha, beta, gamma,
68         ale):
69     T, C = Told.copy(), Cold.copy()
70     for iteration in range(120):
71         cap, k, D = properties(T, C, model, mode, scales)
72         D = moisture_faces(T, C, model, mode, scales)
73         l, d, u, b, _ = assemble(k, cap, Told, Tprev, R, Ro, Rp, dt, alpha, beta, gamma, 25*scales[2],
74                                 ambient[0], ale)
75         Tnew = thomas(l, d, u, b)
76         l, d, u, b, g = assemble(D, np.ones(len(C)), Cold, Cprev, R, Ro, Rp, dt, alpha, beta, gamma, 8e-7*
77                                 scales[1], ambient[1], ale)
78         Cnew = thomas(l, d, u, b)
79         eT, eC = np.max(np.abs(Tnew-T)), np.max(np.abs(Cnew-C))
80         T = omega*Tnew+(1-omega)*T
81         C = omega*Cnew+(1-omega)*C
82         if eT < 1e-7 and eC < 1e-9:
83             if not np.all(np.isfinite(T)) or not np.all(np.isfinite(C)) or np.min(C) <= 0:
84                 raise ValueError('Invalid solution')
85             return T, C, iteration+1, g
86     raise RuntimeError('Picard did not converge')
87
88 @njit(cache=True)
89 def integrate(n, dt, ambient, radius, model, mode, scales, omega, bdf2, ale, stride, event):
90     T, C = np.full(n, 28.), np.full(n, 2.55)
91     Tp, Cp = T.copy(), C.copy()
92     nsteps = len(radius)-1
93     output = np.empty((nsteps//stride+2, 2*n+2))
94     output[0, 0], output[0, 1] = 0., radius[0]
95     output[0, 2:2+n], output[0, 2+n:] = T, C
96     count, maxit, residual, balance = 1, 0, 0., 0.
97     _, v = geometry(n)
98     integral0 = np.sum(v*C)*radius[0]**2
99     loss, lossP = 0., 0.
100     # The no-ALE equation conserves the reference/material integral, not  $R^2 \cdot C$ .
101     if not ale:
102         integral0 = np.sum(v*C)
103     for j in range(1, nsteps+1):
104         alpha, beta, gamma = (1.5, 2., 0.5) if bdf2 and j > 1 else (1., 1., 0.)
105         R, Ro, Rp = radius[j], radius[j-1], radius[max(0, j-2)]
106         Tn, Cn, it, g = step(T, C, Tp, Cp, R, Ro, Rp, ambient[j], dt, model, mode, scales, omega, alpha, beta
107                             , gamma, ale)
108         maxit = max(maxit, it)

```

```

105     V, Vo, Vp = v*R*R, v*Ro*Ro, v*Rp*Rp
106     if not ale:
107         Vo, Vp = V, V
108     flux = -R*8e-7*scales[1]*(Cn[-1]-ambient[j,1]) + g*Cn[-1]
109     err = np.sum(alpha*V*Cn-beta*Vo*C+gamma*Vp* Cp)-dt*flux
110     residual = max(residual, abs(err)/(integral0 if ale else integral0*R*R))
111     audited_flux = flux if ale else flux/(R*R)
112     lossn = (beta*loss-gamma*lossp+dt*audited_flux)/alpha
113     loss, loss = loss, lossn
114     audited_integral = np.sum(V*Cn) if ale else np.sum(v*Cn)
115     balance = max(balance, abs(audited_integral-integral0-loss)/integral0)
116     if event and np.max(Cn) <= 0.15:
117         # Return left bracket states so Python can evaluate exact interpolants during bisection.
118         return output[:count], j, T, C, Tn, Cn, maxit, residual, balance
119     Tp, Cp, T, C = T, C, Tn, Cn
120     if j % stride == 0:
121         output[count, 0], output[count, 1] = j*dt, R
122         output[count, 2:2+n], output[count, 2+n:] = T, C
123         count += 1
124     return output[:count], nsteps, T, C, T, C, maxit, residual, balance
125
126 def solve(model=MODEL_Q23, moving=False, n=81, dt=1., end=432000., interval=60., event=True,
127         scheme='be', interpolation='pchip', tail=None, mode=0, scales=None, omega=1., ale=None):
128     validate_model(model)
129     # Q4 defaults follow the current material/mean formulation; historical
130     # Eulerian runs must explicitly request ale=True and their boundary tail.
131     ale = model != MODEL_Q4 if ale is None else ale
132     tail = ('mean' if model == MODEL_Q4 else 'last') if tail is None else tail
133     if n < 3 or dt <= 0 or abs(interval/dt-round(interval/dt)) > 1e-9:
134         raise ValueError('Invalid grid or sampling interval')
135     scales = np.ones(4) if scales is None else np.asarray(scales, dtype=float)
136     if scales.shape != (4,) or np.min(scales) <= 0:
137         raise ValueError('Expected four positive scales: D, hm, hT, k')
138     boundary = Boundary(interpolation, tail)
139     times = np.arange(int(end/dt)+1)*dt
140     ambient = boundary.ambient(times)
141     radii = boundary.radius(times) if moving else np.full(len(times), .02)
142     out, j, Tleft, Cleft, T, C, maxit, residual, balance = integrate(n, dt, ambient, radii, model, mode,
143         scales, omega, scheme == 'bdf2', ale, int(round(interval/dt)), event)
144     event_s = None
145     bracket = None
146     if event:
147         if np.max(C) > .15:
148             raise RuntimeError(f'No threshold by {end/3600:g} h')
149         left, lo, hi = (j-1)*dt, 0., dt
150         while hi-lo > .1:
151             mid = (lo+hi)/2
152             R = float(boundary.radius(left+mid)) if moving else .02
153             Tmid, Cmid, _, _ = step(Tleft, Cleft, Tleft, Cleft, R, radii[j-1], radii[j-1], boundary.ambient(
154                 left+mid), mid, model, mode, scales, omega, 1., 1., 0., ale)
155             if np.max(Cmid) <= .15:
156                 hi = mid
157             else:
158                 lo = mid
159         event_s, bracket = left+hi, [left+lo, left+hi]
160     R = float(boundary.radius(event_s)) if moving else .02
161     T, C, _, _ = step(Tleft, Cleft, Tleft, Cleft, R, radii[j-1], radii[j-1], boundary.ambient(event_s),

```

```

160     hi, model, mode, scales, omega, 1., 1., 0., ale)
161     out = np.vstack((out, np.r_[event_s, R, T, C]))
162     meta = dict(model=model, moving=moving, n=n, dt=dt, scheme=scheme, interpolation=interpolation, tail=tail
163             , mode=mode,
164             scales=scales.tolist(), omega=omega, ale=ale, event_s=event_s, event_h=None if event_s is
165             None else event_s/3600,
166             event_bracket_s=bracket, max_picard_iterations=int(maxit), max_step_balance_relative=float(
167             residual),
168             max_cumulative_balance_relative=float(balance),
169             balance_quantity='geometric_integral_with_swept_boundary' if ale else '
170             material_integral_uniform_dry_density',
171             balance_includes_event_bisection=False,
172             final_max_C=float(np.max(C)), radius_extrapolated=bool(event_s and moving and event_s>259200)
173         )
174     return out, meta

```

B.5 问题四边界与几何对照

文件: experiments/q4_release.py。

Listing 5 问题四边界与几何对照

```

1  """Q4 material/mean release and matched full-trajectory comparisons."""
2  from pathlib import Path
3  import sys,json, csv,time
4  import numpy as np
5  sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
6  from solver.fvm_cpu import solve
7  from physics.boundary import Boundary
8  from physics.material import MODEL_Q4
9
10 ROOT=Path(__file__).resolve().parents[1]
11 OUT=ROOT/'results'
12
13 def main():
14     folder=OUT/'q4_scenarios';folder.mkdir(exist_ok=True)
15     cases=[('material_mean',{}),('material_last',{'tail':'last'}),
16           ('material_nominal',{'tail':'nominal'}),('fixed_mean',{'moving':False}),
17           ('eulerian_mean',{'ale':True}),('material_mean_dt1',{'dt':1.}),
18           ('material_mean_grid41',{'n':41})]
19     records=[]
20     for name,overrides in cases:
21         options=dict(model=MODEL_Q4,moving=True,ale=False,tail='mean',n=81,dt=.5,
22                 end=180*3600.,interval=60.,event=True)
23         options.update(overrides)
24         start=time.perf_counter(); data,meta=solve(**options)
25         meta.update(name=name,wall_seconds=time.perf_counter()-start,
26                 boundary_plateau=Boundary(tail=options['tail']).tail.tolist())
27         np.savez_compressed(folder/f'{name}.npz',data=data)
28         (folder/f'{name}.json').write_text(json.dumps(meta,indent=2),encoding='utf-8')
29         if name=='material_mean':
30             np.savez_compressed(OUT/'q4.npz',data=data)
31             (OUT/'q4.json').write_text(json.dumps(meta,indent=2),encoding='utf-8')
32         records.append(meta)
33     print(name,meta['event_h'],flush=True)
34     (OUT/'q4_release_experiments.json').write_text(json.dumps(records,indent=2),encoding='utf-8')

```

```

35 fields=['name','event_h','moving','ale','tail','n','dt','max_cumulative_balance_relative']
36 with (OUT/'q4_ablation.csv').open('w',newline='',encoding='utf-8-sig') as f:
37     w=csv.DictWriter(f,fieldnames=fields);w.writeheader()
38     for r in records:w.writerow({k:r[k] for k in fields})
39
40 if __name__=='__main__':main()

```

B.6 问题四收敛及参数试验

文件: experiments/q4_validation.py。

Listing 6 问题四收敛及参数试验

```

1  """Full Q4 material/mean convergence, interpolation and parameter experiments."""
2  from pathlib import Path
3  import sys,json,csv,time,hashlib
4  import numpy as np
5  sys.path.insert(0,str(Path(__file__).resolve().parents[1]))
6  from solver.fvm_cpu import solve
7  from physics.material import MODEL_Q4
8  ROOT=Path(__file__).resolve().parents[1];OUT=ROOT/'results'
9
10 def main():
11     states=OUT/'q4_validation_states';states.mkdir(exist_ok=True)
12     records=[]
13     cases=[(f'grid_{n}',dict(n=n,dt=1.)) for n in (41,81,161)]
14     cases += [(f'dt_{dt:g}',dict(n=81,dt=dt)) for dt in (4.,2.,.5)]
15     cases += [(f'bdf2',dict(scheme='bdf2',dt=1.)),('linear',dict(interpolation='linear',dt=1.))]
16     for j,p in enumerate(('D','hm','hT','k')):
17         for f in (.8,.9,1.1,1.2):
18             scales=np.ones(4);scales[j]=f
19             cases.append((f'{p}_{f:g}',dict(scales=scales,dt=1.)))
20     for name,override in cases:
21         options=dict(model=MODEL_Q4,moving=True,ale=False,tail='mean',n=81,dt=1.,
22                     end=120*3600.,interval=600.,event=True)
23         options.update(override);start=time.perf_counter();d,m=solve(**options)
24         m.update(name=name,wall_seconds=time.perf_counter()-start,output_interval_s=600.)
25         assert m['ale'] is False and m['tail']=='mean' and m['model']==MODEL_Q4
26         assert m['max_cumulative_balance_relative']<1e-8
27         np.savez_compressed(states/f'{name}.npz',data=d)
28         records.append(m)
29         (OUT/'q4_validation.json').write_text(json.dumps(records,indent=2),encoding='utf-8')
30         print(name,m['event_h'],flush=True)
31     by={r['name']:r for r in records};base=by['grid_81']['event_h']
32     with (OUT/'q4_sensitivity.csv').open('w',newline='',encoding='utf-8-sig') as f:
33         w=csv.writer(f);w.writerow(['parameter','factor','event_h','change_percent','source_case'])
34         for p in ('D','hm','hT','k'):
35             for factor in (.8,.9,1.,1.1,1.2):
36                 name='grid_81' if factor==1 else f'{p}_{factor:g}'
37                 value=by[name]['event_h'];w.writerow([p,factor,value,100*(value/base-1),name])
38     with (OUT/'q4_convergence.csv').open('w',newline='',encoding='utf-8-sig') as f:
39         fields=['name','n','dt','scheme','interpolation','event_h']
40         w=csv.DictWriter(f,fieldnames=fields);w.writeheader()
41         for r in records[:8]:w.writerow({k:r[k] for k in fields})
42     main_meta=json.loads((OUT/'q4.json').read_text())
43     assert by['dt_0.5']['event_s']==main_meta['event_s']

```

```

44     proof=dict(model=MODEL_Q4,ale=False,tail='mean',full_trajectories=len(records),
45               primary_event_match=True,baseline_for_sensitivity='grid_81 (N=81, BE dt=1s)',
46               code_sha256={p:hashlib.sha256((ROOT/p).read_bytes().replace(b'\r\n',b'\n')).hexdigest() for p in
47               ['physics/material.py','physics/boundary.py','solver/fvm_cpu.py','experiments/q4_validation.py']})
48               (OUT/'q4_validation_provenance.json').write_text(json.dumps(proof,indent=2),encoding='utf-8')
49
50 if __name__=='__main__':main()

```

B.7 主结果一致性检查

文件: validation/q4_contract.py。

Listing 7 主结果一致性检查

```

1  """Shared fail-closed contract for the current Q4 primary artifacts."""
2  from pathlib import Path
3  import json
4  import numpy as np
5  ROOT=Path(__file__).resolve().parents[1]
6
7  def require_current_q4(meta):
8      required={'model':4,'moving':True,'ale':False,'tail':'mean',
9               'mode':0,'scales':[1.,1.,1.,1.],
10               'balance_quantity':'material_integral_uniform_dry_density'}
11      for key,value in required.items():
12          if key not in meta or meta[key]!=value:
13              raise ValueError(f'Q4 primary model mismatch: {key} must be {value!r}; got {meta.get(key)!r}')
14
15  def load_current_q4():
16      meta=json.loads((ROOT/'results/q4.json').read_text())
17      require_current_q4(meta)
18      if abs(meta['event_h']-meta['event_s']/3600)>1e-12:
19          raise ValueError('Q4 hours and seconds are inconsistent')
20      data=np.load(ROOT/'results/q4.npz')['data'];n=meta['n']
21      if data.shape[1]!=2*n+2 or data[-1,0]!=meta['event_s']:
22          raise ValueError('Q4 NPZ and metadata do not match')
23      if np.max(data[-1,2+n:])!=meta['final_max_C'] or meta['final_max_C']>.15:
24          raise ValueError('Q4 endpoint or threshold does not match metadata')
25      return data,meta

```

B.8 轴对称端面求解与配对比较

文件: validation/endface_benchmark.py。

Listing 8 轴对称端面求解与配对比较

```

1  """Paired axisymmetric end-face benchmark for the prescribed material Q4 model.
2
3  The axial half-domain is  $0 \leq z \leq L/2$ , with symmetry at  $z=0$ . The
4  radial coordinate  $\xi=r/R(t)$  follows the uniformly shrinking material.
5  Only the benchmark exposes both end faces to the same empirical Robin law
6  as the cylindrical side. This is an explicit comparison assumption.
7  """
8  from pathlib import Path
9  import argparse

```

```

10 import csv
11 import hashlib
12 import json
13 import sys
14 import time
15
16 import numpy as np
17 from numba import njit
18 from scipy.sparse import coo_matrix
19 from scipy.sparse.linalg import spsolve
20
21 ROOT = Path(__file__).resolve().parents[1]
22 sys.path.insert(0, str(ROOT))
23 from physics.material import properties, moisture_faces, MODEL_Q4
24 from physics.boundary import Boundary
25 from solver.fvm_cpu import geometry, solve
26
27
28 def grid(nr, nz):
29     xi = np.linspace(0., 1., nr)
30     u = np.linspace(0., 1., nz)
31     z = .125*(1.-(1.-u)**2)
32     rf, vr = geometry(nr)
33     zf = np.r_[0., .5*(z[:-1]+z[1:]), .125]
34     vz = np.diff(zf)
35     weights = vr[:, None]*vz[None, :]
36     node = np.arange(nr*nz).reshape(nr, nz)
37     a = np.r_[node[:-1].ravel(), node[:, :-1].ravel()]
38     b = np.r_[node[1:].ravel(), node[:, 1:].ravel()]
39     rows = np.r_[node.ravel(), a, b]
40     cols = np.r_[node.ravel(), b, a]
41     return xi, z, rf, vr, vz, weights, rows, cols
42
43
44 @njit(cache=True)
45 def matrices(T, C, Told, Cold, R, dt, rf, vr, vz, z, ambient, ends):
46     nr, nz = T.shape
47     n = nr*nz
48     scales = np.ones(4)
49     cap, k, _ = properties(T.ravel(), C.ravel(), MODEL_Q4, 0, scales)
50     dax = moisture_faces(T.ravel(), C.ravel(), MODEL_Q4, 0, scales)
51     drad = moisture_faces(T.T.copy().ravel(), C.T.copy().ravel(), MODEL_Q4, 0, scales)
52     ne = (nr-1)*nz+nr*(nz-1)
53     dataT, dataC = np.zeros(n+2*ne), np.zeros(n+2*ne)
54     rhsT, rhsC = np.empty(n), np.empty(n)
55     for ir in range(nr):
56         for iz in range(nz):
57             p = ir*nz+iz
58             volume = vr[ir]*vz[iz]
59             dataT[p] = cap[p]*volume/dt
60             dataC[p] = volume/dt
61             rhsT[p] = dataT[p]*Told[ir, iz]
62             rhsC[p] = dataC[p]*Cold[ir, iz]
63     edge = 0
64     for ir in range(nr-1):
65         for iz in range(nz):
66             a, b = ir*nz+iz, (ir+1)*nz+iz

```

```

67     geom = rf[ir+1]*(nr-1)*vz[iz]/(R*R)
68     gt = geom*2*k[a]*k[b]/(k[a]+k[b])
69     gc = geom*drad[iz*nr+ir]
70     dataT[a] += gt
71     dataT[b] += gt
72     dataC[a] += gc
73     dataC[b] += gc
74     dataT[n+edge] = dataT[n+ne+edge] = -gt
75     dataC[n+edge] = dataC[n+ne+edge] = -gc
76     edge += 1
77     for ir in range(nr):
78         for iz in range(nz-1):
79             a, b = ir*nz+iz, ir*nz+iz+1
80             geom = vr[ir]/(z[iz+1]-z[iz])
81             gt = geom*2*k[a]*k[b]/(k[a]+k[b])
82             gc = geom*dax[a]
83             dataT[a] += gt
84             dataT[b] += gt
85             dataC[a] += gc
86             dataC[b] += gc
87             dataT[n+edge] = dataT[n+ne+edge] = -gt
88             dataC[n+edge] = dataC[n+ne+edge] = -gc
89             edge += 1
90     for iz in range(nz):
91         p = (nr-1)*nz+iz
92         st, sc = 25.*vz[iz]/R, 8e-7*vz[iz]/R
93         dataT[p] += st
94         dataC[p] += sc
95         rhsT[p] += st*ambient[0]
96         rhsC[p] += sc*ambient[1]
97     if ends:
98         for ir in range(nr):
99             p = ir*nz+nz-1
100             st, sc = 25.*vr[ir], 8e-7*vr[ir]
101             dataT[p] += st
102             dataC[p] += sc
103             rhsT[p] += st*ambient[0]
104             rhsC[p] += sc*ambient[1]
105     return dataT, dataC, rhsT, rhsC
106
107
108 def step(Told, Cold, R, dt, ambient, ends, mesh):
109     _, z, rf, vr, vz, weights, rows, cols = mesh
110     T, C = Told.copy(), Cold.copy()
111     n = C.size
112     for it in range(120):
113         dataT, dataC, rhsT, rhsC = matrices(T, C, Told, Cold, R, dt, rf, vr, vz, z, ambient, ends)
114         mt = coo_matrix((dataT, (rows, cols)), shape=(n, n)).tocsc()
115         mc = coo_matrix((dataC, (rows, cols)), shape=(n, n)).tocsc()
116         tn = spsolve(mt, rhsT).reshape(T.shape)
117         cn = spsolve(mc, rhsC).reshape(C.shape)
118         et, ec = np.max(np.abs(tn-T)), np.max(np.abs(cn-C))
119         T, C = tn, cn
120         if et < 1e-7 and ec < 1e-9:
121             if not np.isfinite(T).all() or not np.isfinite(C).all() or np.min(C) <= 0:
122                 raise ValueError('Invalid nonlinear solution')
123             return T, C, it+1

```

```

124     raise RuntimeError('2D Picard did not converge')
125
126
127 def run(nr=41, nz=25, dt=30., ends=True, label='coarse'):
128     started = time.perf_counter()
129     mesh = grid(nr, nz)
130     xi, z, _, vr, vz, weights, _, _ = mesh
131     ref_volume = weights.sum()
132     boundary = Boundary('pchip', 'mean')
133     T, C = np.full((nr, nz), 28.), np.full((nr, nz), 2.55)
134     snapshots = {0.: (T.copy(), C.copy())}
135     history = [[0., .02, 2.55, 2.55, 28., 28., 2.55, 2.55, 0., 0.]]
136     times = np.arange(0., 216000*dt/2, dt)
137     ambients, radii = boundary.ambient(times), boundary.radius(times)
138     max_step, max_total, maxit = 0., 0., 0
139     side_loss, end_loss = 0., 0.
140     event_bracket = None
141     total_steps = 0
142     for j in range(1, len(times)):
143         t, tj, cj = times[j], T, C
144         T, C, it = step(tj, cj, radii[j], dt, ambients[j], ends, mesh)
145         maxit = max(maxit, it)
146         actual_dt = dt
147         if C.max() <= .15:
148             lo, hi = 0., dt
149             while hi-lo > .1:
150                 mid = .5*(lo+hi)
151                 tm = t-dt+mid
152                 tt, cc, _ = step(tj, cj, float(boundary.radius(tm)), mid, boundary.ambient(tm), ends, mesh)
153                 if cc.max() <= .15:
154                     hi = mid
155                 else:
156                     lo = mid
157             event_bracket = [float(t-dt+lo), float(t-dt+hi)]
158             actual_dt, t = hi, t-dt+hi
159             T, C, it = step(tj, cj, float(boundary.radius(t)), actual_dt, boundary.ambient(t), ends, mesh)
160             R = float(boundary.radius(t))
161             ambient = boundary.ambient(t)
162             fs = 8e-7/R*np.sum(vz*(C[-1]-ambient[1]))/ref_volume
163             fe = 8e-7*np.sum(vr*(C[:, -1]-ambient[1]))/ref_volume if ends else 0.
164             oldmean, mean = np.sum(weights*cj)/ref_volume, np.sum(weights*C)/ref_volume
165             side_loss += actual_dt*fs
166             end_loss += actual_dt*fe
167             max_step = max(max_step, abs(mean-oldmean+actual_dt*(fs+fe))/2.55)
168             max_total = max(max_total, abs(mean-2.55+side_loss+end_loss)/2.55)
169             total_steps += 1
170             if any(abs(t-s) < 1e-8 for s in (1800., 14400., 86400.)) or event_bracket:
171                 snapshots[float(t)] = (T.copy(), C.copy())
172             if j % max(1, int(600/dt)) == 0 or event_bracket:
173                 history.append([t, R, C.max(), mean, T.min(), T.max(), C[0, 0], C[0, -1], side_loss, end_loss])
174             if j % max(1, int(21600/dt)) == 0:
175                 print(f'{label}: {t/3600:.1f} h, maxC={C.max():.6f}, wall={time.perf_counter()-started:.1f}s',
176                       flush=True)
177             if event_bracket:
178                 break
179     if not event_bracket:
180         raise RuntimeError('No event by 60 h')

```

```

180     result = dict(label=label, model=MODEL_Q4, formulation='material_coordinate', tail='mean',
181                  nr=nr, nz=nz, dt_s=dt, scheme='BE', ends=ends,
182                  axial_half_length_m=.125, end_hm_m_s=8e-7 if ends else 0.,
183                  end_hT_W_m2_K=25. if ends else 0.,
184                  z_grid='z=0.125*(1-(1-u)^2), uniform u in [0,1]',
185                  event_h=t/3600, event_s=t, event_bracket_s=event_bracket,
186                  mean_C_final=float(mean), max_C_final=float(C.max()),
187                  side_loss_C_average=float(side_loss), end_loss_C_average=float(end_loss),
188                  normalized_dry_mass=1.,
189                  normalized_water_mass_definition='Mw/Md = sum(vr*vz*C)/(0.5*L/2)',
190                  max_step_water_budget_relative=float(max_step),
191                  max_cumulative_water_budget_relative=float(max_total),
192                  budget_includes_final_event_step=True, max_picard_iterations=maxit,
193                  steps=total_steps, wall_seconds=time.perf_counter()-started,
194                  snapshots=[])
195     for ts, (tt, cc) in snapshots.items():
196         result['snapshots'].append(dict(time_s=ts, max_C=float(cc.max()),
197                                       mean_C=float(np.sum(weights*cc)/ref_volume),
198                                       center_midplane_C=float(cc[0, 0]),
199                                       center_end_C=float(cc[0, -1]),
200                                       min_T=float(tt.min()), max_T=float(tt.max())))
201     path = ROOT/'results'/f'endface_{label}'
202     path.with_suffix('.json').write_text(json.dumps(result, ensure_ascii=False, indent=2)+'\n', encoding='utf-8')
203     np.savez_compressed(path.with_suffix('.npz'), xi=xi, z=z, weights=weights,
204                       times=np.array(list(snapshots)),
205                       T=np.array([x[0] for x in snapshots.values()]),
206                       C=np.array([x[1] for x in snapshots.values()]), history=np.array(history))
207     print(json.dumps(result, ensure_ascii=False), flush=True)
208     return result
209
210
211 def paired_1d(nr, dt, label):
212     started = time.perf_counter()
213     out, meta = solve(model=MODEL_Q4, moving=True, n=nr, dt=dt, interval=600.,
214                      ale=False, tail='mean', scheme='be')
215     meta['wall_seconds'] = time.perf_counter()-started
216     path = ROOT/'results'/f'endface_{label}_1d'
217     path.with_suffix('.json').write_text(json.dumps(meta, ensure_ascii=False, indent=2)+'\n', encoding='utf-8')
218     np.savez_compressed(path.with_suffix('.npz'), data=out)
219     print(json.dumps(meta, ensure_ascii=False), flush=True)
220
221
222 def summarize():
223     import matplotlib
224     import scipy
225     import numba
226     matplotlib.use('Agg')
227     import matplotlib.pyplot as plt
228
229     records, details = [], {}
230     for label in ('closed', 'coarse', 'fine'):
231         base = ROOT/'results'/f'endface_{label}'
232         two = json.loads(base.with_suffix('.json').read_text(encoding='utf-8'))
233         one = json.loads((ROOT/'results'/f'endface_{label}_1d.json').read_text(encoding='utf-8'))
234         d2 = np.load(base.with_suffix('.npz'))

```

```

235 d1 = np.load(ROOT/'results'/f'endface_{label}_1d.npz')['data']
236 nr = two['nr']
237 _, vr = geometry(nr)
238 paired = []
239 for idx, ts in enumerate(d2['times']):
240     j = int(np.argmin(abs(d1[:, 0]-ts)))
241     if abs(d1[j, 0]-ts) > 1e-8:
242         if ts == two['event_s']:
243             # Do not compare fields evaluated at different event clocks.
244             continue
245             raise AssertionError('Missing exact representative 1D time')
246     t1, c1 = d1[j, 2:2+nr], d1[j, 2+nr:]
247     t2, c2 = d2['T'][idx], d2['C'][idx]
248     mean1 = float(2*np.sum(vr*c1))
249     mean2 = float(np.sum(d2['weights']*c2)/np.sum(d2['weights']))
250     paired.append(dict(time_s=float(ts), max_C_1d=float(c1.max()), max_C_2d=float(c2.max()),
251                       max_C_difference=float(c2.max()-c1.max()),
252                       mean_C_1d=mean1, mean_C_2d=mean2,
253                       mean_C_difference=mean2-mean1,
254                       max_abs_T_field_vs_repeated_1d=float(np.max(abs(t2-t1[:, None]))),
255                       max_abs_C_field_vs_repeated_1d=float(np.max(abs(c2-c1[:, None]))))
256     record = dict(label=label, nr=nr, nz=two['nz'], dt_s=two['dt_s'], ends=two['ends'],
257                  event_1d_h=one['event_h'], event_2d_h=two['event_h'],
258                  event_difference_s=two['event_s']-one['event_s'],
259                  final_mean_C_2d=two['mean_C_final'],
260                  final_mean_C_1d=float(2*np.sum(vr*d1[-1, 2+nr:])),
261                  max_cumulative_water_budget_relative=two['max_cumulative_water_budget_relative'],
262                  end_fraction_of_integrated_loss=two['end_loss_C_average']/(two['end_loss_C_average']+
263                  two['side_loss_C_average']))
264     records.append(record)
265     details[label] = paired
266     closed = details['closed']
267     if max(r['max_abs_T_field_vs_repeated_1d'] for r in closed) > 1e-8:
268         raise AssertionError('Closed-end temperature failed 1D degeneration')
269     if max(r['max_abs_C_field_vs_repeated_1d'] for r in closed) > 1e-9:
270         raise AssertionError('Closed-end moisture failed 1D degeneration')
271     if records[0]['event_difference_s'] != 0.:
272         raise AssertionError('Closed-end event failed 1D degeneration')
273     result = dict(scope='Conditional engineering end-face benchmark; not a change to frozen Q4 main results',
274                  runtime=dict(python=sys.version, numpy=np.__version__, scipy=scipy.__version__, numba=numba
275                  .__version__),
276                  assumptions=[
277                      'Same hT and hm on each end and cylindrical side',
278                      'Fixed L=0.25m and uniform material radial shrinkage',
279                      'Independent conserved dry-solid density; empirical rho*cp only in thermal
280                      equation',
281                      'Identical effective equilibrium moisture mapping Ceq=Cair'],
282                  rows=records, representative_time_pairs=details,
283                  closed_end_degeneracy_passed=True,
284                  paired_event_difference_change_s=records[2]['event_difference_s']-records[1]['
285                  event_difference_s'],
286                  sources_sha256={str(p.relative_to(ROOT)): hashlib.sha256(p.read_bytes()).hexdigest() for p
287                  in
288                      [Path(__file__), ROOT/'physics/material.py', ROOT/'physics/boundary.py',
289                      ROOT/'solver/fvm_cpu.py', ROOT/'data/boundary.csv', ROOT/'data/radius.csv'
290                      ]})
291     (ROOT/'results/endface_summary.json').write_text(json.dumps(result, ensure_ascii=False, indent=2)+'\n',
292     encoding='utf-8')

```

```

285 with (ROOT/'results/endface_summary.csv').open('w', encoding='utf-8-sig', newline='') as f:
286     w = csv.DictWriter(f, fieldnames=records[0].keys())
287     w.writeheader()
288     w.writerows(records)
289 fig, axes = plt.subplots(1, 2, figsize=(12, 4.4), constrained_layout=True)
290 d2 = np.load(ROOT/'results/endface_fine.npz')
291 d1 = np.load(ROOT/'results/endface_fine_1d.npz')['data']
292 _, vr = geometry(records[-1]['nr'])
293 n = records[-1]['nr']
294 axes[0].plot(d1[:, 0]/3600, d1[:, 2+n:].max(axis=1), color='#1b4f72', label='1D max C')
295 axes[0].plot(d2['history'][:, 0]/3600, d2['history'][:, 2], '--', color='#e67e22', label='2D max C')
296 axes[0].plot(d1[:, 0]/3600, 2*(d1[:, 2+n:]@vr), color='#1b4f72', alpha=.6, label='1D mean C')
297 axes[0].plot(d2['history'][:, 0]/3600, d2['history'][:, 3], '--', color='#c0392b', label='2D mean C')
298 axes[0].axhline(.15, color='gray', lw=.8)
299 axes[0].set(xlabel='Time (h)', ylabel='C (kg water/kg dry solid)', title='Paired 1D / 2D moisture
    histories')
300 axes[0].legend(fontsize=8)
301 for idx, ts in enumerate(d2['times']):
302     if ts == 0.:
303         continue
304     axes[1].plot(d2['z']*100, d2['C'][idx, 0], label=f'{ts/3600:.2f} h')
305 axes[1].set(xlabel='z from midplane (cm)', ylabel='Axis C (kg/kg)', title='Axial profile at radial center
    ')
306 axes[1].legend(fontsize=8)
307 for ax in axes:
308     ax.grid(alpha=.2)
309 fig.savefig(ROOT/'results/endface_comparison.png', dpi=180)
310 fig.savefig(ROOT/'results/endface_comparison.svg')
311 plt.close(fig)
312 fine = records[-1]
313 baseline = json.loads((ROOT/'results/q4.json').read_text(encoding='utf-8'))
314 closed_t = max(x['max_abs_T_field_vs_repeated_1d'] for x in details['closed'])
315 closed_c = max(x['max_abs_C_field_vs_repeated_1d'] for x in details['closed'])
316 lines = ['\n## 实际结果与解释\n',
317         '| 场景 | Nr x Nz | dt/s | 配对1D时间/h | 2D时间/h | 2D-1D/s | ',
318         '|---|---:|---:|---:|---:|---:|']
319 for x in records:
320     lines.append(f"| {x['label']} | {x['nr']} x {x['nz']} | {x['dt_s']:.0f} | {x['event_1d_h']:.9f} | {x
321         ['event_2d_h']:.9f} | {x['event_difference_s']:.6f} |")
322 lines += [' ', f'关闭端面的完整二维场与重复的配对 1D 场一致：代表时刻及事件时刻最大温度差 {closed_t:.3e} K，最大
    含水率差 {closed_c:.3e} kg/kg；事件时间一致。', ' ',
323         '  细级开放端面对照如下。平均量采用干固体质量归一化，所有表中比较都在相同时刻进行。', ' ',
324         '| 时刻/h | 1D max C | 2D max C | 1D mean C | 2D mean C | 均值相对改变 | ',
325         '|---:|---:|---:|---:|---:|---:|']
326 for x in details['fine']:
327     if x['time_s'] == 0.:
328         continue
329     change = 100*x['mean_C_difference']/x['mean_C_1d']
330     lines.append(f"| {x['time_s']/3600:.6f} | {x['max_C_1d']:.9f} | {x['max_C_2d']:.9f} | {x['mean_C_1d
331         ']:.9f} | {x['mean_C_2d']:.9f} | {change:.4f}% |")
332 coarse_delta = records[1]['final_mean_C_2d']-records[1]['final_mean_C_1d']
333 fine_delta = fine['final_mean_C_2d']-fine['final_mean_C_1d']
334 lines += [' ', f"细级全程归一化水质量预算最大相对误差为 {fine['max_cumulative_water_budget_relative']:.3e}（包
    含事件短步）；累计端面排水占全部排水 {100*fine['end_fraction_of_integrated_loss']:.4f}%。粗、细级终态平均 C 的
    配对改变量分别为 {coarse_delta:.9f}、{fine_delta:.9f} kg/kg。", ' ',
335         f"f较大步长的细级配对 1D 时间比主结果 {baseline['event_h']:.9f} h 晚 {(fine['event_1d_h']-baseline
    ['event_h'])*3600:.3f} s。这一绝对差主要属于所用离散设置，不能解释为端面影响，二维数值也没有替换主结果。", ' ',

```

```

334         '两个分辨率的配对事件结果应结合各 JSON 中的事件括区读取。若括区重合，只表明当前离散问题没有分辨出相应尺
        度的时间变化；这不是连续 PDE 端面误差的严格上界，也不消除剩余网格、时间步、外部边界与端面参数的不确定性。', '',
335         '**在本例规定长度、均匀径向变形和相同端面 Robin 参数下，端面对全域最大含水率达标时刻的影响很小；但对平均
        含水率和端部局部温湿场有可见影响。** 因此本验证支持针对中心阈值时间使用 1D 近似，不能支持“整个温湿场的端面误差均
        可忽略”。', '',
336         '![配对历史与轴向剖面](../results/endface_comparison.png)', ''
337     document = ROOT/'validation/端面验证.md'
338     prose = document.read_text(encoding='utf-8').split('\n## 实际结果与解释')[0]
339     document.write_text(prose+'\n'+'\n'.join(lines), encoding='utf-8')
340     print(json.dumps(result, ensure_ascii=False, indent=2))
341
342
343 if __name__ == '__main__':
344     parser = argparse.ArgumentParser()
345     parser.add_argument('--nr', type=int, default=41)
346     parser.add_argument('--nz', type=int, default=25)
347     parser.add_argument('--dt', type=float, default=30.)
348     parser.add_argument('--label', default='coarse')
349     parser.add_argument('--closed-ends', action='store_true')
350     parser.add_argument('--paired-1d', action='store_true')
351     parser.add_argument('--summarize', action='store_true')
352     args = parser.parse_args()
353     if args.summarize:
354         summarize()
355     else:
356         run(args.nr, args.nz, args.dt, not args.closed_ends, args.label)
357     if args.paired_1d:
358         paired_1d(args.nr, args.dt, args.label)

```

B.9 独立数值验证

文件：tests/test_solver.py。

Listing 9 独立数值验证

```

1 import json
2 from pathlib import Path
3 import sys
4 sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
5 import numpy as np
6 from scipy.linalg import solve_banded
7 from scipy.optimize import brentq
8 from scipy.special import j0, j1
9 from scipy.integrate import quad
10 from solver.fvm_cpu import assemble, thomas, solve
11 from physics.material import moisture_faces, properties, MODEL_Q4
12 from validation.q4_contract import require_current_q4
13
14 def main():
15     n = 41
16     x = np.linspace(0, 1, n)
17     args = (np.full(n, .36), np.full(n, 820*2600.), np.full(n, 28.), np.full(n, 28.), .019, .02, .02, 1., 1.,
18           1., 0., 25., 50., True)
19     l, d, u, b, _ = assemble(*args)
20     band = np.zeros((3,n)); band[0,1:] = u; band[1] = d; band[2,:-1] = 1
21     tri_error = float(np.max(np.abs(thomas(1,d,u,b)-solve_banded((1,1), band,b))))

```

```

21 assert tri_error < 1e-10
22 # Geometric conservation, contraction AND expansion, BE AND BDF2.
23 constant_error = 0.
24 for R in (.019, .021):
25     for a, beta, gamma in ((1.,1.,0.), (1.5,2.,.5)):
26         l,d,u,b,_ = assemble(np.ones(n), np.ones(n), np.full(n,2.), np.full(n,2.), R,.02,.02,1.,a,beta,
27                               gamma,0.,2.,True)
28         constant_error = max(constant_error, float(np.max(np.abs(thomas(l,d,u,b)-2))))
29 assert constant_error < 1e-7
30 # Single exact Robin eigenmode: u=J0(lambda*r/R)*exp(-alpha*lambda^2*t/R^2).
31 Bi, R, diffusivity = 25*.02/.36, .02, .36/(820*2600)
32 lam = brentq(lambda z:z*j1(z)-Bi*j0(z), .01, 2.4048)
33 errors = []
34 for n in (21,41,81):
35     x = np.linspace(0,1,n); old = j0(lam*x); dt=.05
36     for _ in range(2000):
37         l,d,u,b,_ = assemble(np.full(n,.36),np.full(n,820*2600.),old,old,R,R,R,dt,1.,1.,0.,25.,0.,True)
38         old = thomas(l,d,u,b)
39         exact = j0(lam*x)*np.exp(-diffusivity*lam**2*100/R**2)
40         errors.append(float(np.max(np.abs(old-exact))))
41 assert errors[2] < errors[1] < errors[0] and errors[2] < 5e-5
42 out, meta = solve(model=1,n=21,dt=1,end=1800,interval=1,event=False)
43 assert np.all(out[:,2:23] >= 28-1e-7) and np.all(out[:,23:] > 0)
44 assert out[-1,22] > out[-1,2] and out[-1,-1] < out[-1,23]
45 assert meta['max_cumulative_balance_relative'] < 1e-6
46 # A shrinking material mesh must audit the normalized dry-basis integral.
47 # Previously this path silently returned an unchecked zero.
48 material_checks = {}
49 for scheme in ('be', 'bdf2'):
50     _, check = solve(model=MODEL_Q4, moving=True, ale=False, n=41, dt=2.,
51                     end=14400., interval=60., event=False, scheme=scheme)
52     assert check['balance_quantity'] == 'material_integral_uniform_dry_density'
53     assert check['max_cumulative_balance_relative'] < 1e-8
54     material_checks[scheme] = check['max_cumulative_balance_relative']
55 # Invalid identifiers must never silently select the Q4 formula.
56 for bad in (0, 3, 5, 999):
57     for function in (properties, moisture_faces):
58         try:
59             function(np.full(3,28.),np.full(3,2.55),bad,0,np.ones(4))
60         except ValueError:
61             pass
62         else:
63             raise AssertionError(f'{function} accepted invalid model {bad}')
64 _, default_q4 = solve(model=MODEL_Q4,moving=True,n=21,dt=2.,end=1800.,event=False)
65 assert default_q4['ale'] is False and default_q4['tail']=='mean'
66 # A stale Eulerian JSON must be rejected by the export/report contract.
67 for patch in ({'ale':True},{ 'tail':'last'},{ 'model':3},{ 'moving':False},{ 'mode':1},{ 'scales'
68               :[1.2,1.,1.,1.]}) :
69     stale=dict(default_q4);stale.update(patch)
70     try:
71         require_current_q4(stale)
72     except ValueError:
73         pass
74     else:
75         raise AssertionError(f'Stale primary metadata accepted: {patch}')
76 # Independent adaptive quadrature validates nonlinear face transmissibility.
77 quadrature_error = 0.

```

```

76     for c1,c2 in ((.05,.1),(.1,.4),(2.,2.55)):
77         T,C=np.array([45.,50.]),np.array([c1,c2])
78         actual=moisture_faces(T,C,2,0,np.ones(4))[0]
79         exact=quad(lambda s:2.4e-3*np.exp(-.45/(c1*(1-s)+c2*s))-3850/(45*(1-s)+50*s+273.15)),0,1,epsabs=1e-22)
80         [0]
81         quadrature_error=max(quadrature_error,abs(actual-exact)/exact)
82     assert quadrature_error<1e-6
83     result = dict(thomas_scipy_max_error=tri_error, geometric_constant_error=constant_error,
84                   cylindrical_eigenmode_errors=errors, nonlinear_face_quadrature_error=quadrature_error,
85                   material_coordinate_balance=material_checks, q1=meta)
86     dest = Path(__file__).resolve().parents[1]/'results'; dest.mkdir(exist_ok=True)
87     (dest/'tests.json').write_text(json.dumps(result, indent=2),encoding='utf-8')
88     print(json.dumps(result,indent=2))
89 if __name__ == '__main__': main()

```

B.10 Excel 数值准备

文件: export/prepare_outputs.py。

Listing 10 Excel 数值准备

```

1  from pathlib import Path
2  import json
3  import hashlib
4  import sys
5  import numpy as np
6
7  ROOT=Path(__file__).resolve().parents[1]
8  sys.path.insert(0,str(ROOT))
9  from validation.q4_contract import load_current_q4
10
11 def main():
12     payload=[]
13     load_current_q4()
14     for q in range(1,5):
15         data=np.load(ROOT/'results'/f'q{q}.npz')['data']
16         meta=json.loads((ROOT/'results'/f'q{q}.json').read_text())
17         n=meta['n']; data=data[data[:,0]>0]
18         radii=np.arange(21)/10
19         sheets=[]
20         for title,start in (('温度',2),('水分浓度',2+n)) if q<=2 else (('Sheet1',2+n)):
21             values=[]
22             for row in data:
23                 field=row[start:start+n]; r=row[1]*100
24                 sampled=np.interp(radii,np.linspace(0,r,n),field)
25                 entries=[round(float(v),4) if pos<=r+1e-10 else None for pos,v in zip(radii,sampled)]
26                 if q==4: entries.append(round(float(field[-1]),4))
27                 values.append([float(row[0])]+entries)
28             header=[ '时间\\到药材中心的距离 ']+radii.tolist()+(['药材表面' if q==4 else []])
29             sheets.append(dict(name=title,rows=[header]+values))
30             item=dict(q=q,sheets=sheets)
31             if q==4:
32                 item['source_sha256']={f'q4.{ext}':hashlib.sha256((ROOT/'results'/f'q4.{ext}').read_bytes()).hexdigest() for ext in ('json','npz')}
33             payload.append(item)

```

```

34     if q==4:
35         np.savetxt(ROOT/'results'/'q4_radius_output.csv',np.column_stack((data[:,0],data[:,1]*100)),
36             delimiter=',',header='time_s,radius_cm',comments='')
37         (ROOT/'results'/'workbooks.json').write_text(json.dumps(payload,ensure_ascii=False,allow_nan=False),
38             encoding='utf-8')
39
40 if __name__=='__main__':main()

```

B.11 Excel 模板导出

文件: export/workbooks.mjs。

Listing 11 Excel 模板导出

```

1  import fs from 'node:fs/promises';
2  import path from 'node:path';
3  import {createHash} from 'node:crypto';
4  import {fileURLToPath} from 'node:url';
5  import {FileBlob,SpreadsheetFile} from '@oai/artifact-tool';
6  const root=path.resolve(path.dirname(fileURLToPath(import.meta.url)), '..');
7  const out=path.join(root,'results');
8  const preview=path.join(out,'previews');await fs.mkdir(preview,{recursive:true});
9  const mode=process.argv[2]??'export';
10 if(mode==='templates'){
11     for(let q=1;q<=4;q++){
12         const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(path.join(root,'data','templates`,`result${
13             q}.xlsx`)));
14         const sheet=wb.worksheets.getItemAt(0);
15         const blob=await wb.render({sheetName:sheet.name,range:'A1:F5',scale:1.5,format:'png'});
16         await fs.writeFile(path.join(preview`,`template${q}.png`),new Uint8Array(await blob.arrayBuffer()));
17     }
18 }else{
19     const books=JSON.parse(await fs.readFile(path.join(out,'workbooks.json'),'utf8'));
20     for(const book of books.filter(b=>!process.argv[3] || b.q===Number(process.argv[3]))){
21         if(book.q===4){
22             if(!book.source_sha256) throw new Error('Regenerate the Q4 payload with current model checks.');
```

```

41     sheet.freezePanes.freezeRows(1);
42 }
43 wb.recalculate();
44 for(const spec of book.sheets){
45     const blob=await wb.render({sheetName:spec.name,range:'A1:H8',scale:1.2,format:'png'});
46     await fs.writeFile(path.join(preview,`result${book.q}_${spec.name}.png`),new Uint8Array(await blob.
arrayBuffer()));
47     if(book.q===4){
48         const last=spec.rows.length;
49         for(const [label,range] of [['end',`A${last-4}:F${last}`],[`surface`,`R${last-4}:W${last}`]]){
50             const snapshot=await wb.render({sheetName:spec.name,range,scale:1.2,format:'png'});
51             await fs.writeFile(path.join(preview,`result4_${label}.png`),new Uint8Array(await snapshot.
arrayBuffer()));
52         }
53     }
54     console.log((await wb.inspect({kind:'table',range:`${spec.name}`!A1:D3`,include:'values',tableMaxRows
:3,tableMaxCols:4,maxChars:600})).ndjson);
55 }
56 console.log((await wb.inspect({kind:'match',searchTerm:'#REF!|#DIV/0!|#VALUE!|#NAME\\?|#NUM!',options:{
useRegex:true,maxResults:5,maxChars:400})).ndjson);
57 await (await SpreadsheetFile.exportXlsx(wb)).save(path.join(out,`result${book.q}.xlsx`));
58 console.log(`Exported result${book.q}.xlsx`);
59 }
60 }

```

B.12 论文图与结果表生成

文件: paper/prepare_paper_assets.py。

Listing 12 论文图与结果表生成

```

1  """Generate manuscript figures and tables from the existing numerical results."""
2  from pathlib import Path
3  import csv
4  import hashlib
5  import json
6  import sys
7  import numpy as np
8  import matplotlib
9  matplotlib.use('Agg')
10 import matplotlib.pyplot as plt
11 from matplotlib import font_manager
12
13 PAPER = Path(__file__).resolve().parent
14 ROOT = PAPER.parent
15 sys.path.insert(0, str(ROOT))
16 from physics.boundary import Boundary
17
18 FIG = PAPER / 'figures'
19 TAB = PAPER / 'tables'
20 FIG.mkdir(exist_ok=True)
21 TAB.mkdir(exist_ok=True)
22 font_manager.fontManager.addfont(str(PAPER/'完整论文-LaTeX/fonts/SimHei.ttf'))
23 plt.rcParams.update({'font.family':'SimHei','font.size':9,'axes.titlesize':10,
24     'axes.labelsize':9,'legend.fontsize':8,'axes.unicode_minus':False,
25     'pdf.fonttype':42,'ps.fonttype':42,'savefig.dpi':360,'lines.linewidth':1.55,

```

```

26     'axes.spines.top':False, 'axes.spines.right':False,
27     'axes.prop_cycle':plt.cycler(color=['#176B87', '#D07935', '#4F8460', '#9A5B75', '#5261A6'])))
28 COL=['#176B87', '#D07935', '#4F8460', '#9A5B75', '#5261A6']
29
30 def save(fig, name):
31     fig.savefig(FIG/f'{name}.pdf', bbox_inches='tight')
32     fig.savefig(FIG/f'{name}.png', bbox_inches='tight', dpi=360)
33     plt.close(fig)
34
35 def grid(ax):
36     ax.grid(alpha=.18)
37     ax.set_axisbelow(True)
38
39 def load(q):
40     d=np.load(ROOT/'results'/f'q{q}.npz')['data']
41     n=(d.shape[1]-2)//2
42     faces=np.r_[0, (np.arange(n-1)+.5)/(n-1), 1]
43     v=np.diff(faces**2)/2
44     return d, n, v
45
46 b=Boundary(tail='mean')
47 t=np.linspace(0, 14400, 1000)
48 a=b.ambient(t)
49 fig, axs=plt.subplots(1, 2, figsize=(6.25, 2.55), layout='constrained')
50 for j, ax in enumerate(axs):
51     ax.plot(t/3600, a[:, j], label='PCHIP 插值')
52     ax.scatter(b.data[:, 6, 0]/3600, b.data[:, 6, j+1], s=6, color=COL[1], label='观测值 (每6点显示1点)', zorder=3)
53     ax.set(xlabel='时间 / h', ylabel='温度 / °C' if j==0 else '附件水分变量 / (kg/kg)', title='(a) 烘房温度' if j
54           ==0 else '(b) 烘房水分变量')
55     ax.axvspan(3, 4, color=COL[2], alpha=.08)
56     grid(ax)
57     axs[0].legend(loc='lower right', frameon=False)
58     save(fig, 'boundary')
59
60 fig, axs=plt.subplots(1, 2, figsize=(6.25, 2.45), layout='constrained')
61 tr=np.linspace(0, 72, 1200)
62 rr=b.radius(tr*3600)
63 axs[0].plot(tr, rr*100, label='PCHIP 插值')
64 axs[0].scatter(b.radius_data[:, 4, 0]/3600, b.radius_data[:, 4, 1], s=6, c=COL[1], label='观测值 (间隔2h显示)')
65 axs[0].axvline(51.0912847222, color=COL[2], ls='--', lw=1)
66 axs[0].set(xlabel='时间 / h', ylabel='半径 / cm', title='(a) 附件2半径轨迹')
67 axs[0].legend(frameon=False)
68 axs[1].plot(tr, .02**2/rr**2, label='扩散系数几何倍率 (R0/R)^2')
69 axs[1].plot(tr, .02/rr, label='边界系数几何倍率 R0/R')
70 axs[1].set(xlabel='时间 / h', ylabel='相对初始状态的倍率', title='(b) 归一化方程中的几何作用')
71 axs[1].legend(frameon=False, loc='lower right')
72 for ax in axs: grid(ax)
73 save(fig, 'radius')
74
75 for q, times in [(1, [100, 600, 1200, 1800]), (2, [1800, 3600, 7200, 10800])]:
76     d, n, v=load(q)
77     fig, axs=plt.subplots(1, 2, figsize=(6.25, 2.55), layout='constrained')
78     for time in times:
79         row=d[np.argmin(abs(d[:, 0]-time))]
80         label=f'{time:g} s' if q==1 else f'{time/3600:g} h'
81         axs[0].plot(np.linspace(0, 2, n), row[2:2+n], label=label)
82         axs[1].plot(np.linspace(0, 2, n), row[2+n:], label=label)

```

```

82     axs[0].set(ylabel='温度 / °C',title='(a) 温度的径向分布')
83     axs[1].set(ylabel='干基含水率 / (kg/kg)',title='(b) 含水率的径向分布')
84     for ax in axs:
85         ax.set_xlabel('到轴线的距离 / cm')
86         ax.legend(frameon=False,ncol=2)
87         grid(ax)
88     save(fig,f'q{q}_fields')
89
90 for q in [3,4]:
91     d,n,v=load(q);c=d[:,2+n:]
92     fig,axs=plt.subplots(1,2,figsize=(6.25,2.6),layout='constrained')
93     axs[0].semilogy(d[:,0]/3600,c.max(axis=1),label='最大含水率')
94     axs[0].semilogy(d[:,0]/3600,2*c@v,label='干质量加权平均')
95     axs[0].semilogy(d[:,0]/3600,c[:,-1],label='侧表面')
96     axs[0].axhline(.15,c='#777777',ls='--',lw=.8,label='达标阈值0.15')
97     axs[0].set_yticks([.05,.1,.15,.3,1,2.55],labels=['0.05','0.1','0.15','0.3','1','2.55'])
98     axs[0].yaxis.set_minor_formatter(matplotlib.ticker.NullFormatter())
99     axs[0].set(xlabel='时间 / h',ylabel='干基含水率 / (kg/kg)',title='(a) 干燥过程 (纵轴为对数)')
100    axs[0].legend(frameon=False)
101    for time in [6*3600,12*3600,24*3600,d[-1,0]]:
102        row=d[np.argmin(abs(d[:,0]-time))]
103        axs[1].plot(np.linspace(0,row[1]*100,n),row[2+n:],label=f'{time/3600:.2f} h')
104    axs[1].set(xlabel='到轴线的距离 / cm',ylabel='干基含水率 / (kg/kg)',title='(b) 代表时刻的物理坐标剖面')
105    axs[1].legend(frameon=False)
106    for ax in axs:grid(ax)
107    save(fig,f'q{q}_drying')
108
109 d,n,v=load(4);c=d[:,2+n:]
110 ratio=(d[:,1]/.02)**2*(2*((760+90*c)/(1+c))@v)/((760+90*2.55)/(1+2.55))
111 statistics={'q4_implied_dry_mass_min':float(ratio.min()),'q4_implied_dry_mass_min_time_h':float(d[ratio.
112     argmin(),0]/3600),
113     'q4_implied_dry_mass_final':float(ratio[-1]),'q4_final_mean_C':float(2*c[-1]@v)}
114 fig,axs=plt.subplots(1,2,figsize=(6.25,2.45),layout='constrained')
115 axs[0].plot(d[:,0]/3600, (.02/d[:,1])**2,label='独立干密度 s/s0')
116 axs[0].plot(d[:,0]/3600,np.ones(len(d)),ls='--',label='真实干质量 Md/Md0')
117 axs[0].set(xlabel='时间 / h',ylabel='相对初始值',title='(a) 随体收缩模型的质量定义')
118 axs[0].legend(frameon=False)
119 axs[1].plot(d[:,0]/3600,ratio,c=COL[1])
120 axs[1].axhline(1,ls='--',c='#777777',lw=.8)
121 axs[1].scatter([d[ratio.argmin(),0]/3600],[ratio.min()],s=17,c=COL[1])
122 axs[1].annotate(f'最低 {ratio.min():.4f}',xy=(d[ratio.argmin(),0]/3600,ratio.min()),xytext=(18,.78),arrowprops
123     ={'arrowstyle':'->','color':'#555555'},fontsize=8)
124 axs[1].set(xlabel='时间 / h',ylabel='推算干质量 / 初始推算干质量',title='(b) 额外令  $s=\rho(C)/(1+C)$  的矛盾')
125 for ax in axs:grid(ax)
126 save(fig,'q4_mass_definition')
127
128 cases=json.loads((ROOT/'results/q4_release_experiments.json').read_text())
129 by={r['name']:r for r in cases}
130 fig,axs=plt.subplots(1,2,figsize=(6.25,2.4),layout='constrained')
131 labels=['末点保持','末小时均值','名义平台']
132 values=[by[n]['event_h'] for n in ['material_last','material_mean','material_nominal']]
133 axs[0].barh(labels,values,color=[COL[1],COL[0],COL[2]],height=.55)
134 axs[0].set(xlim=(0,58),xlabel='干燥时长 / h',title='(a) 材料坐标的三种边界情景')
135 for i,val in enumerate(values):axs[0].text(val+.5,i,f'{val:.4f}',va='center',fontsize=8)
136 axs[1].barh(['固定半径','随体收缩','Eulerian对照'],[by[n]['event_h'] for n in ['fixed_mean','material_mean','
137     eulerian_mean']],color=[COL[1],COL[0],COL[2]],height=.55)
138 axs[1].set(xlim=(0,151),xlabel='干燥时长 / h',title='(b) 同物性、同均值平台对照')

```

```

136 for i,name in enumerate(['fixed_mean','material_mean','eulerian_mean']):
137     val=by[name]['event_h'];axs[1].text(val+1.5,i,f'{val:.4f}',va='center',fontsize=8)
138 for ax in axs:ax.invert_yaxis();ax.grid(axis='x',alpha=.18);ax.set_axisbelow(True)
139 save(fig,'q4_scenarios_ablation')
140
141 rows=list(csv.DictReader((ROOT/'results/q4_convergence.csv').open(encoding='utf-8-sig')))
142 bc={r['name']:r for r in rows}
143 fig,axs=plt.subplots(1,2,figsize=(6.25,2.45),layout='constrained')
144 x=np.array([41,81,161]);y=np.array([float(bc[f'grid_{n}']['event_h']) for n in x])
145 axs[0].plot(x,(y-y[-1])*3600,'o-')
146 axs[0].set(xlabel='径向节点数 N',ylabel='相对 N=161 的时差 / s',xticks=x,title='(a) 空间加密 (时间步长1s)')
147 x=np.array([.5,1.,2.,4.]);y=np.array([float(bc[k]['event_h']) for k in ['dt_0.5','grid_81','dt_2','dt_4']])
148 axs[1].plot(x,(y-y[0])*3600,'o-')
149 axs[1].set(xlabel='时间步长 / s',ylabel='相对步长0.5s的时差 / s',xticks=x,title='(b) 时间加密 (81节点)')
150 for ax in axs:grid(ax)
151 save(fig,'q4_convergence')
152
153 rows=list(csv.DictReader((ROOT/'results/q4_sensitivity.csv').open(encoding='utf-8-sig')))
154 fig,axs=plt.subplots(1,2,figsize=(6.25,2.5),layout='constrained')
155 for p,label in [(('D',r'$D$'),('hm',r'$h_m$'),('hT',r'$h_T$'),('k',r'$k$'))]:
156     rr=[r for r in rows if r['parameter']==p]
157     x=[float(r['factor']) for r in rr];y=[float(r['change_percent']) for r in rr]
158     axs[0].plot(x,y,'o-',label=label,ms=3)
159     if p=='D':axs[1].plot(x,y,'o-',label=label,ms=3)
160 for ax in axs:
161     ax.set(xlabel='参数倍率',ylabel='干燥时长变化 / %',xticks=[.8,.9,1,1.1,1.2])
162     ax.legend(frameon=False);grid(ax)
163 axs[0].set_title('(a) 全部参数的相对影响')
164 axs[1].set_title('(b) 其余三个参数的局部放大')
165 save(fig,'q4_sensitivity')
166
167 en=json.loads((ROOT/'results/endface_summary.json').read_text())
168 rr=en['representative_time_pairs']['fine'][1:]
169 fig,axs=plt.subplots(1,2,figsize=(6.25,2.5),layout='constrained')
170 x=np.arange(len(rr));w=.34
171 axs[0].bar(x-w/2,[r['mean_C_1d'] for r in rr],w,label='一维',color=COL[0])
172 axs[0].bar(x+w/2,[r['mean_C_2d'] for r in rr],w,label='二维开放端面',color=COL[1])
173 axs[0].set(xticks=x,xticklabels=['0.5','4','24','51.10'],xlabel='时间 / h',ylabel='平均干基含水率 / (kg/kg)',
174           title='(a) 同设置配对的平均含水率')
175 axs[0].legend(frameon=False)
176 endata=np.load(ROOT/'results/endface_fine.npz')
177 statistics['endface_npz_shapes']={k:list(endata[k].shape) for k in endata.files}
178 delta=[100*(r['mean_C_2d']/r['mean_C_1d']-1) for r in rr]
179 axs[1].bar(x,delta,color=COL[1],width=.55)
180 for i,val in enumerate(delta):axs[1].text(i,val-.06,f'{val:.2f}%',ha='center',va='top',fontsize=8)
181 axs[1].set(xticks=x,xticklabels=['0.5','4','24','51.10'],xlabel='时间 / h',ylabel='二维相对一维的变化 / %',
182           title='(b) 平均含水率对端面交换的响应',ylim=(-3.8,0))
183 for ax in axs:ax.grid(axis='y',alpha=.18);ax.set_axisbelow(True)
184 save(fig,'endface')
185
186 # The six requested tables: generate all displayed values from authoritative CSV.
187 captions=[
188     '30分钟内药材的温度 (单位: $^\circ\text{C}$)',
189     '30分钟内药材的干基含水率 (单位: kg/kg)',
190     '3小时内药材的温度 (单位: $^\circ\text{C}$)',
191     '3小时内药材的干基含水率 (单位: kg/kg)',
192     '固定半径药材烘干过程的干基含水率 (单位: kg/kg)',
193     '随体收缩药材烘干过程的干基含水率 (单位: kg/kg)']
194 for i in range(1,7):
195     rows=list(csv.reader((ROOT/'results'/f'table{i}.csv').open(encoding='utf-8-sig')))

```

```

191 headers=rows[0];body=rows[1:]
192 # CSV uses a textual first column; every remaining numerical entry has 4 decimals.
193 nc=len(headers)
194 colspec='r'+ 'r'*(nc-1)
195 first='时间 / s' if i<=2 else '时间 / h'
196 h=[first,'0 cm','0.5 cm','1 cm','1.5 cm','2 cm']
197 if i==6:h+=['侧面','半径 / cm']
198 text=['\\begin{table}[htbp]',f'\\caption{{{captions[i-1]}}}\\label{{tab:required-{i}}}',
199       '\\centering',('\\small' if i!=6 else '\\footnotesize'),f'\\begin{{tabular}}{{{colspec}}}', '\\
200       toprule',' & '.join(h)+r' \\',r'\\midrule']
201 for row in body:
202     vals=[]
203     for j,vv in enumerate(row):
204         if vv in ('','—','nan'):vals.append('---')
205         elif j==0:vals.append(f'{float(vv):.4f}' if i>=5 and float(vv)%6 else f'{float(vv):g}')
206         else:vals.append(f'{float(vv):.4f}')
207     text.append(' & '.join(vals)+r' \\')
208 text += [r'\\bottomrule',r'\\end{tabular}',r'\\end{table}']
209 (TAB/f'table{i}.tex').write_text('\\n'.join(text)+'\\n',encoding='utf-8')
210
211 statistics['assets_sources']={str(p.relative_to(ROOT)):hashlib.sha256(p.read_bytes()).hexdigest() for p in
212 [ROOT/'results'/f'q{q}.npz' for q in range(1,5)]+[ROOT/'results/q4_convergence.csv',ROOT/'results/
213 q4_sensitivity.csv',ROOT/'results/endface_summary.json']}
214 (PAPER/'asset_statistics.json').write_text(json.dumps(statistics,indent=2,ensure_ascii=False),encoding='utf-8')
215 print(json.dumps(statistics,ensure_ascii=False,indent=2))

```